

Contents

Implementation Plan: GPU Support for SEAS-MFEM (Dynamic + Quasi-Dynamic) — INCITE Edition	3
V1.1 → V2 changes summary	3
Overview	4
Constraints	4
Interface constraints (must not change)	4
Dependency constraints	4
Convention constraints	4
Numerical / performance constraints	5
Failure mode policy (carried from project CLAUDE.md)	6
Target hardware matrix [V2]	6
Aurora gap [V2]	7
Background math (referenced by Phase 3/4/5)	8
Velocity–stress system	8
Godunov flux at interior face	9
Rate-and-state friction (residual form solved per fault DOF)	9
Cauchy–Kovalevskaya recursion (ADER predictor)	9
Phase 0: Build infrastructure + Device::Configure plumbing	9
Goal	9
Files to Create	9
Files to Modify	11
Detailed Requirements	12
Interfaces	14
Edge Cases to Handle	14
Acceptance Criteria	14
Dependencies	15
Phase 1: BP5 sparse solve on GPU via HYPRE	15
Goal	15
Files to Create	15
Files to Modify	15
Detailed Requirements	16
Interfaces	16
Edge Cases to Handle	16
Acceptance Criteria	16
Dependencies	17
Phase 2: Friction-solver kernels on GPU (BP5 + dynamic rupture)	17
Goal	17
Files to Create	17
Files to Modify	17
Detailed Requirements	19
Interfaces	19
Edge Cases to Handle	19
Acceptance Criteria	20
Dependencies	20
Phase 3: Wave-operator element-local kernels (volume + mass inverse + spatial derivative)	20
Goal	20
Files to Create	20
Files to Modify	21
Detailed Requirements	22
Interfaces	24
Edge Cases to Handle	24
Acceptance Criteria	25
Dependencies	25
Phase 4: Wave-operator face-flux kernels on GPU	25

Goal	25
Files to Create	25
Files to Modify	26
Detailed Requirements	27
Interfaces	28
Edge Cases to Handle	28
Acceptance Criteria	28
Dependencies	28
Phase 5: ADER predictor–corrector on GPU	29
Goal	29
Files to Create	29
Files to Modify	29
Detailed Requirements	29
Interfaces	30
Edge Cases to Handle	30
Acceptance Criteria	30
Dependencies	30
Phase 6: BP5 RHS rebuild + traction recovery on GPU	30
Goal	30
Files to Create	31
Files to Modify	31
Detailed Requirements	31
Interfaces	31
Edge Cases to Handle	32
Acceptance Criteria	32
Dependencies	32
Phase 7: Profiling, validation, performance tuning, and INCITE readiness [V2]	32
Goal	32
Files to Create	32
Files to Modify	33
Detailed Requirements	33
Interfaces	34
Edge Cases to Handle	34
Acceptance Criteria	34
Dependencies	34
Dependencies	35
PART II — Cross-cutting concerns	35
Testing Strategy	35
Risk Assessment	35
Out of scope (deferred to follow-on plans)	37
Appendix A: file-level inventory of changes by phase	37
Appendix B: kernel LOC budget	38
Appendix C: kernel-level data residency contract	38
Appendix D: INCITE node-hour budget [V2]	40
Per-platform node-hour conversion	40
Development node-hours per phase (estimate)	40
Production node-hours for the INCITE proposal	42
Suggested INCITE ask	42
Re-cost trigger	42

Implementation Plan: GPU Support for SEAS-MFEM (Dynamic + Quasi-Dynamic) — INCITE Edition

Author: code-plan agent Date: 2026-05-16 (v1 + v1.1 + v2) Target branch: feature/gpu-port (off claude/priceless-carson-f1ea1b) Predecessor (kept on disk): gpu_implementation_plan_v1.md (v1.1, post-fix) Analysis seed: gpu_porting_analysis_2026-05-16.md Supersedes: dynamic_rupture_plan_v4.md §Phase 6 (was sketchy / never executed) Review status (v1.1): 18 review findings from gpu_implementation_plan_v1_checked incorporated; each section tagged with [R-GPU-NNN]. Fix report: gpu_implementation_plan_v1_fix.md. v2 deltas (this document) are tagged inline [V2] and summarized below.

V1.1 → V2 changes summary

v1.1 assumed a single development GPU (NVIDIA A100). v2 retargets the plan for an INCITE allocation proposal spanning DOE leadership systems:

Site	System	GPU	Programming model	INCITE?
OLCF	Frontier	AMD MI250X (2 GCDs/GPU, 4 GPUs/node = 8 GCDs/node)	HIP / ROCm	yes
ALCF	Aurora	Intel Data Center GPU Max (Ponte Vecchio, 2 stacks/GPU, 6 GPUs/node = 12 stacks/node)	SYCL / oneAPI	yes — gated, see §“Aurora gap”
ALCF	Polaris	NVIDIA A100 (4 GPUs/node)	CUDA	yes
OLCF	Summit	NVIDIA V100 (6 GPUs/node, CC 7.0)	CUDA	yes

Concrete v2 deltas:

1. New §“Target hardware matrix” (after Constraints) — pins per-platform compiler / MPI / HYPRE / CUDA-arch / module tuples.
2. Numerical / performance constraints rewritten as a per-platform table; targets per device, not just A100.
3. Phase 0 build infrastructure now produces four platform build scripts (build_polaris.sh, build_frontier_olcf.sh, build_summit.sh, build_aurora.sh — last one gated on an MFEM CEED-SYCL bridge that does not currently exist; see §“Aurora gap”). The existing repo script is build_frontera.sh (TACC Frontera, a CPU system) — do not conflate with OLCF Frontier.
4. Phase 7 baselines expanded to four-column per-platform table; adds an INCITE Readiness deliverable (weak + strong scaling demo on Polaris + Frontier minimum, Aurora if gap closes, Summit for V100-class validation).

5. Risk assessment adds four new rows: Aurora SYCL gap, Summit V100 CC-7.0 atomic-CAS cost, MI250X dual-GCD partitioning, per-site HYPRE-GPU version drift.
6. New Appendix D: INCITE node-hour budget — ballpark estimate of development node-hours per phase and production node-hours for the proposal.
7. §“Aurora gap” callout (immediately after the hardware matrix) spells out the blocker and the decision tree: CEED-SYCL bridge, upstream MFEM SYCL, OpenMP fallback on Aurora Xeon Max CPU cores, or drop Aurora from the proposal.

v2 changes do NOT alter any Phase 1–6 kernel algorithms or interfaces. All [R–GPU–NNN] review fixes remain in force.

Overview

Add CUDA / HIP GPU execution paths to the SEAS-MFEM-paraview miniapp covering both quasi-dynamic (BP5) and dynamic-rupture (TPV102 / TPV104 / TPV205) benchmarks. The plan keeps the existing CPU code paths byte-identical and adds a runtime-selectable device path (`--device cuda|hip|cpu`) gated by `mfem::Device::Configure`. The plan does not attempt to port custom DG integrators to MFEM's PA path (blocked by tet-mesh + vector-elasticity PA gap); instead, it hand-writes `mfem::forall` kernels for the ~8 hot routines that dominate the time-step cost while reusing the full MFEM infrastructure (Vector, Mesh, FiniteElementSpace, L2FaceRestriction, GeometricFactors, HYPRE) for everything else.

Constraints

Interface constraints (must not change)

- Public APIs of `WaveOperator`, `ElasticityDomainOperator`, `RateStateFaultOperator`, `SEASQuasi-DynamicOperator`, `DomainOperator` base class, `FaultBasis`, `FaultGeometry`, `FrictionSolver`, `DieterichRuinaFriction`, `GodunovFlux`, `FaultFaceFlux`, `BoundaryConfig`, `DomainConfig`.
- TOML config schema, driver CLI surface (only additive changes — new `--device` flag, optional `[runtime].device = "cuda"` block).
- Existing CPU regression tests must remain bit-identical when `--device cpu` is selected explicitly.

Dependency constraints

- HYPRE $\geq 2.31.0$ built with `--with-cuda` (NVIDIA) or `--with-hip` (AMD) for runtime-selectable GPU policy (see `INSTALL:647–651`).
- MFEM built with `MFEM_USE_CUDA=YES` (or `MFEM_USE_HIP=YES`) + `MFEM_USE_MPI=YES` + `MFEM_USE_LAPACK=YES`.
- CUDA Toolkit ≥ 11.8 (or ROCm ≥ 6.0) matching HYPRE / MFEM versions.
- Optional: CUDA-aware MPI (OpenMPI ≥ 4.1 with `--with-cuda` or MPICH with `--with-cuda`) to keep `ParGridFunction::ExchangeFaceNbrData` device-resident.
- PETSc (when `use_petsc_ts = true` in TOML) must be built with CUDA support and `--with-cuda-arch` matching the target SM.

Convention constraints

- Follow the codebase's existing patterns:
 - All public headers in `dynamic/`, `domain/`, `fault/`, `friction/`.
 - Tests in `tests/unit/test_<feature>.cpp` (CPU/GPU equivalence), and `tests/parallel/test_<feature>_np<np>.cpp` ($np \geq 2$).
 - Markdown debug docs in `bp5_debug_document/` for BP5 work, `dynamic_rupture_debug_document/` for TPV (create if absent).
 - Issue tags follow existing scheme: R–GPU–NNN for review notes, G–NNN for new GPU-specific issue IDs.

- New kernel files: `dynamic/kernels/<topic>_kernels.hpp/.cpp` to keep them separate from the CPU `wave_operator.inl`.
- Every `mfem::forall` kernel must:
 - Use `Reshape(vec.Read(), ...)` for inputs, `Reshape(vec.Write(), ...)` or `Reshape(vec.ReadWrite(), ...)` for outputs (no `GetData()`).
 - Tag captured functions with `MFEM_HOST_DEVICE`.
 - INSIDE the kernel body (the lambda passed to `mfem::forall`): no `std::vector`, `std::map`, `std::sort`, virtual dispatch, exceptions. Setup code on the host may use these freely. [R-GPU-018]
 - Treat `DenseMatrix::Mult` etc. as forbidden — use raw scalar loops.
 - For accumulation into shared output slots (face flux scatter, reductions), use `mfem::AtomicAdd<real_t>(slot, val)` from `general/backends.hpp`. Never use raw CUDA / HIP `atomicAdd` — only `mfem::AtomicAdd<T>` is portable across CUDA, HIP, and the OpenMP host fallback. [R-GPU-005]

Numerical / performance constraints

- CPU↔GPU equivalence:
 - Element-local kernels (volume, mass-inverse, `ApplySpatialDerivative`, friction loop): max-norm relative error $< 1e-12$ over a single `Mult` / `AdvanceADER` call vs CPU reference.
 - Face-flux kernels: max-norm relative error $< 1e-11$ (loose tolerance accommodates FP-order non-determinism from face traversal order).
 - Full driver run (TPV102 1 second, BP5 100 s): `chunhui-benchmark` PASS at the existing CPU tolerances.
- HYPRE PCG+AMG GPU vs CPU on identical matrix: iteration counts may differ by up to 2× (different AMG smoother — `l1Jacobi` on GPU vs `l1GS` on CPU per `linalg/hypre.cpp:1504,1840,...`); final solution L2 relative diff $< 1e-6$ at PCG tolerance $1e-8$. Tighter solution tolerance (e.g., $1e-10$) requires PCG tol $1e-12$ and an explicit `SetPrintLevel` audit — out of scope for Phase 1 acceptance. [R-GPU-007]
- Target speedup, per platform (single device vs single CPU socket on the same node, BP5 500m for the linear-solve / end-to-end rows, TPV102 200m for the dynamic rows). Targets are conservative; Phase 7 is allowed to revise them based on profile data. [V2]

Metric	Polaris A100	Frontier MI250X (per GCD)	Summit V100	Aurora PVC (per stack)
End-to-end BP5 500m	$\geq 3\times$	$\geq 2.5\times$	$\geq 1.5\times$	TBD (Aurora gap)
HYPRE PCG+AMG solve	$\geq 5\times$	$\geq 4\times$	$\geq 2\times$	TBD
Friction NR loop	$\geq 8\times$	$\geq 6\times$	$\geq 4\times$	TBD
Volume kernel (TPV102)	$\geq 6\times$	$\geq 5\times$	$\geq 3\times$	TBD
Face flux (atomic-add)	$\geq 4\times$	$\geq 3\times$	$\geq 1.2\times$ (atomic-CAS)	TBD
ADER predictor (TPV104)	$\geq 5\times$	$\geq 4\times$	$\geq 2\times$	TBD

Rationale for V100 lower bars: Volta (CC 7.0) has no hardware double-precision `atomicAdd` — MFEM's `general/backends.hpp:69–91` CAS-loop fallback applies. The face-flux atomic path is the worst-case here (~5–10× CAS overhead under contention); other kernels do not atomic and scale closer to the FP64 peak ratio. [V2]

- Memory budget, per device (single rank). State vector + 8 ADER scratch buffers at TPV102 default mesh ≈ 12 GiB at order 4. The smallest target is V100 (16 GiB), which constrains the largest per-rank problem size — TPV102 production runs targeting Summit must use at least 4 $\times$ domain decomposition vs Frontier/Polaris.

Device	HBM	Effective per-rank cap
Polaris A100 (80GB SKU)	80 GiB	64 GiB
Frontier MI250X (per GCD)	64 GiB	50 GiB
Aurora PVC (per stack)	64 GiB	50 GiB
Summit V100 (16 GiB SKU)	16 GiB	12 GiB

Per-platform cap drives the minimum np for each system; see Appendix D budget. [V2]

Failure mode policy (carried from project CLAUDE.md)

- Missing GPU kernel must MFEM_ABORT with a clear message naming the routine and the offending mode, NOT silently fall back to a host implementation that pretends to be GPU. The user has explicitly required this in past sessions.
- Build failures (missing HYPRE GPU, mismatched CUDA arch, etc.) must fail at cmake / make time with descriptive errors, never at runtime.

Target hardware matrix [V2]

Each row is a (site, system, device, programming model) tuple that the plan must support in build-system flags, runtime backends, and Phase 7 performance baselines. The “module / env” column records the per-site toolchain version that the corresponding Phase 0 build script must load.

Site	Compute System	GPU device + arch flag	Programming model + MFEM backend	HYPRE GPU?	MPI flavour	Module / env (representative)	Notes
ALCF	Polaris 1 $\times$ AMD Milan + 4 $\times$ NVIDIA A100 (40 GB HBM2)	sm_80	CUDA + MFEM_USE_CUDA, Backend::CUDA	yes ($\geq$ 2.31, --with-cuda)	Cray MPICH (GPU-aware via MPICH_GPU_SUPPORT_ENABLED)	module load PrgEnv-nvhpc cudatoolkit-standalone/12.x cray-hdf5- 2.10.1	Reference platform; closest development laptop expectations.

Site	Compute Systemnode	GPU device + arch flag	Programming model + MFEM backend	HYPRE GPU?	MPI flavour	Module / env (representative)	Notes
OLCF	Frontier1× AMD EPYC + 4× MI250X (8 GCDs @ 64 GB HBM2e each)	gfx90a	HIP + MFEM_USE_HIP, Backend::HIP	yes (≥ 2.31, -- with-hip)	Cray MPICH (GPU-aware via MPICH_GPU_SUPPORTS_ENABLED=1)	module load PrgEnv-amd/1.10.0-20200527-craype-accel-amd-gfx90a parallel	Treat each MI250X GCD as a separate “GPU” — 8 GCDs/node, 8 MPI ranks/node. Blocked on MFEM main-line; see Aurora gap.
ALCF	Aurora 2× Intel Xeon Max + 6× PVC (12 stacks @ 64 GB HBM2e each)	pvc (Level Zero bdf:0/bdf:1)	SYCL / oneAPI + no MFEM backend yet — see §“Aurora gap”	experimental (HYPRE SYCL via Ginkgo / Kokkos, not via main-line)	intel MPI / MPICH (GPU-aware via ZE_FLAT_DEVICE_HIERARCHY=COMPOSITE)	module load oneapi/eng-compiler intel-pti hdf5/<v>	Blocked on MFEM main-line; see Aurora gap.
OLCF	Summit2× IBM POWER9 + 6× V100 (16 GB HBM2)	sm_70	CUDA + MFEM_USE_CUDA, Backend::CUDA	yes (≥ 2.31, -- with-cuda)	Spectrum MPI (GPU-aware via -- smpiargs="--gpu" jsrun flag)	module load cuda/12.x gcc/12 spectrum-mpi hdf5/<v>	V100 lacks fast FP64 atomi-cAdd (CC 7.0) — see Risk Assessment.

Programming-model invariant. Phase 1–6 kernels are written once against MFEM's `mfem::forall + Vector::Read/Write` API. MFEM dispatches the same kernel source through CUDA on Polaris/Summit, through HIP on Frontier, and through (eventually) a SYCL bridge on Aurora. The application code does NOT contain CUDA/HIP-specific `#ifdefs` — those live inside MFEM. The build scripts (Phase 0) select the backend at MFEM configure time.

Aurora gap [V2]

Status: Aurora support is BLOCKED at the time of writing.

The blocker: `general/device.hpp:33–79` enumerates MFEM’s backend list. It contains CUDA, HIP, RAJA_CUDA, RAJA_HIP, OCCA_CUDA, CEED_CUDA, CEED_HIP, and DEBUG_DEVICE. There is no SYCL backend, no **CEED_SYCL** backend, and no Level Zero backend. A `grep` of MFEM’s `general/`, `linalg/`, and `config/` directories returns zero matches for `sycl` / SYCL. HYPRE’s SYCL execution policy (≥ 2.32 , experimental) is reachable through `Hypre::InitDevice` but only if the surrounding MFEM device context understands a SYCL backend — which it does not.

Aurora porting options, ranked by realism:

1. CEED-SYCL via a new MFEM **Backend::CEED_SYCL** (most realistic). libCEED’s SYCL backend exists and runs on Aurora PVC (oneAPI DevSummit 2023 report). Adding CEED_SYCL to MFEM mirrors the existing CEED_CUDA / CEED_HIP pattern. However, our Phase 3–5 kernels are not written through CEED’s QFunction API — they are hand-rolled `mfem::forall` kernels. To dispatch them on Aurora through CEED, we’d have to re-author the kernels as CEED QFunctions. This is a ~Phase 4–6 worth of rework (~2–3 months on top of v1.1’s existing estimate).
2. Upstream MFEM SYCL backend. Out of our hands; MFEM project has not announced a SYCL backend roadmap as of 2026-05. Hope, but do not plan on, this happening.
3. OpenMP-on-Xeon-Max fallback on Aurora. Aurora compute nodes carry 2× Xeon Max CPUs with HBM. Running our `--device cpu` path on Aurora’s CPU cores uses the node’s HBM but ignores the 6 PVC GPUs. This is a Plan-B fallback for the INCITE proposal: we can still claim “we run on Aurora” using CPU-only.
4. Drop Aurora. Submit INCITE for Polaris + Frontier (+ Summit) only. Acceptable if reviewers do not penalize platform breadth.

Decision (this plan): Phase 0 produces a stub `build_aurora.sh` script that configures MFEM in CPU-only mode (`MFEM_USE_CUDA=N0`, `MFEM_USE_HIP=N0`) so we can at least demonstrate the BP5 / TPV benchmarks on Aurora’s Xeon Max CPUs. Per-platform performance table in §“Numerical / performance constraints” leaves Aurora PVC entries as TBD (Aurora gap). Phase 7 §“INCITE Readiness deliverable” includes an Aurora-CPU scaling demo separately from the GPU demos.

Re-evaluation trigger: if the INCITE program officer rejects the proposal for “insufficient Aurora support” or MFEM upstream lands a SYCL backend before the resubmission window, escalate options 1 or 2 into a separate Phase 8 plan.

Background math (referenced by Phase 3/4/5)

Velocity–stress system

For the 3D linear isotropic elastic-wave equation in conservation form, the state is $\mathbf{Q} = (\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z)^T \in \mathbb{R}^9$ and

$$\partial_t \mathbf{Q} + \mathbf{A}_x \partial_x \mathbf{Q} + \mathbf{A}_y \partial_y \mathbf{Q} + \mathbf{A}_z \partial_z \mathbf{Q} = 0$$

with constant Jacobians $\mathbf{A}_d \in \mathbb{R}^{9 \times 9}$ (material-only). DG weak form (strong-form variant used in `ComputeVolumeRHS`):

$$\int_K \mathbf{Q}_t \cdot \mathbf{v} dV = \sum_{d=0}^2 \int_K (\partial_{x_d} \mathbf{v}) \cdot \mathbf{A}_d \mathbf{Q} dV - \int_{\partial K} \hat{\mathbf{F}} \cdot \mathbf{n} \cdot \mathbf{v} dS.$$

The volume-integral kernel computes $\text{rhs}_{c,i,e} += \sum_q w_q \sum_d (\partial_{x_d} \phi_i)(x_q) [\mathbf{A}_d \mathbf{Q}(x_q)]_c$.

Godunov flux at interior face

For neighbour states Q^-, Q^+ , unit normal $\hat{n}$, rotate to face frame, split into characteristic contributions, and recombine:

$$\hat{F}(Q^-, Q^+, \hat{n}) = T \cdot \frac{1}{2} (A_n^+ Q_R^- + A_n^- Q_R^+)$$

with T the 9×9 rotation matrix, $Q_R^\pm = T^{-1} Q^\pm$, $A_n^\pm$ the positive/negative-eigenvalue parts of the rotated Jacobian. Implementation: `dynamic/godunov_flux.cpp`.

Rate-and-state friction (residual form solved per fault DOF)

$$g(V) \equiv |\sigma_n| a \sinh^{-1} \left(\frac{V}{2V_0} e^{\psi/a} \right) + \eta_s V - \Theta = 0$$

Brent (CPU default), Newton (GPU default), Newton-Stable (SolveSlipRateNewtonStable, TPV104 canonical). See `friction_solver.hpp:38–112`.

Cauchy–Kovalevskaya recursion (ADER predictor)

For ADER order O :

$$D^{(0)} = Q, \quad D^{(k+1)} = - \sum_d A_d \partial_{x_d} D^{(k)} \text{ (element-local),} \quad I = \int_0^{\Delta t} Q(t + \tau) d\tau = \sum_{k=0}^{O-1} \frac{\Delta t^{k+1}}{(k+1)!} D^{(k)}.$$

Implementation: `wave_operator.inl:1004–1099`.

Phase 0: Build infrastructure + **Device::Configure** plumbing

Goal

After Phase 0: every existing benchmark binary builds on each INCITE-target platform (Polaris/CUDA, Frontier/HIP, Summit/CUDA, Aurora/CPU-fallback), accepts a `--device <cuda|hip|cpu>` CLI flag, and produces byte-identical results to the pre-change main when `--device cpu` is selected. No physics changes yet. [V2]

Files to Create

Four platform build scripts, mirroring the pattern of the existing `build_frontera.sh` (which targets TACC Frontera, a Skylake CPU system — do not confuse with OLCF Frontier). Each script: (a) loads the site modules, (b) verifies required environment variables, (c) calls `make config` with site-specific flags, (d) builds MFEM + the four drivers. All four scripts live at the repo root (sibling to `build_frontera.sh`), not in `miniapps/seas/`, because they configure the MFEM library itself. [V2]

- **build_polaris.sh** — Polaris (ALCF, NVIDIA A100, CUDA):

```
module load PrgEnv-nvhpc cudatoolkit-standalone/12.x \
           cray-hdf5-parallel cray-libsci
# Build HYPRE locally with: ./configure --with-cuda \
#   --with-cuda-home=$CUDAToolkit_HOME --with-MPI ...
export CUDA_ARCH=sm_80
make config \
  MFEM_USE_MPI=YES MFEM_USE_CUDA=YES MFEM_USE_METIS=YES \
  MFEM_USE_LAPACK=YES \
```

```
CUDA_ARCH=$CUDA_ARCH \
HYPRE_OPT="-I${HYPRE_DIR}/include" \
HYPRE_LIB="-L${HYPRE_DIR}/lib -lHYPRE -lcusparse -lcublas -lcurand -lcusolver"
make -j16 && cd miniapps/seas && make -j16
```

- **build_frontier_olcf.sh** — Frontier (OLCF, AMD MI250X, HIP):

```
module load PrgEnv-amd amd/<ROCM-ver> craype-accel-amd-gfx90a \
        cray-hdf5-parallel cray-libsci
export HIP_ARCH=gfx90a
export MPICH_GPU_SUPPORT_ENABLED=1 # GPU-aware MPI on Cray MPICH
# Build HYPRE locally with: ./configure --with-hip \
# --with-MPI-include=${MPICH_DIR}/include ...
make config \
MFEM_USE_MPI=YES MFEM_USE_HIP=YES MFEM_USE_METIS=YES \
MFEM_USE_LAPACK=YES \
HIP_ARCH=$HIP_ARCH \
HYPRE_OPT="-I${HYPRE_DIR}/include" \
HYPRE_LIB="-L${HYPRE_DIR}/lib -lHYPRE -L${ROCM_PATH}/lib -lrocspase -lroclblas -lrocrand"
make -j16 && cd miniapps/seas && make -j16
```

- **build_summit.sh** — Summit (OLCF, NVIDIA V100, CUDA):

```
module load cuda/12.x gcc/12 spectrum-mpi hdf5/<v>
export CUDA_ARCH=sm_70 # V100; lacks fast FP64 atomicAdd
make config \
MFEM_USE_MPI=YES MFEM_USE_CUDA=YES MFEM_USE_METIS=YES \
MFEM_USE_LAPACK=YES \
CUDA_ARCH=$CUDA_ARCH \
HYPRE_OPT="-I${OLCF_HYPRE_ROOT}/include" \
HYPRE_LIB="-L${OLCF_HYPRE_ROOT}/lib -lHYPRE"
make -j16 && cd miniapps/seas && make -j16
```

Build script must include a WARNING: echo at the top noting that Summit V100 (CC 7.0) uses MFEM's CAS-loop FP64 atomicAdd fallback — face-flux kernel performance will be lower than CC 6.0+ targets.

- **build_aurora.sh** — Aurora (ALCF, Intel PVC, CPU-only fallback pending §“Aurora gap” resolution):

```
module load oneapi/eng-compiler intel-pti hdf5/<v>
# NO MFEM_USE_CUDA / MFEM_USE_HIP - CPU-only on Xeon Max
# MFEM_USE_SYCL does not exist in mainline MFEM as of 2026-05.
make config \
MFEM_USE_MPI=YES MFEM_USE_OPENMP=YES MFEM_USE_METIS=YES \
MFEM_USE_LAPACK=YES \
CXX=mpiicpx CC=mpiicx \
HYPRE_OPT="-I${HYPRE_DIR}/include" \
HYPRE_LIB="-L${HYPRE_DIR}/lib -lHYPRE"
make -j16 && cd miniapps/seas && make -j16
```

Header comment in this script must explicitly state Aurora GPU support is BLOCKED on the §“Aurora gap” resolution; the script produces a CPU-only binary that runs on Xeon Max but ignores the PVC devices. Update this script when CEED_SYCL or MFEM SYCL lands.

- miniapps/seas/common/device_init.hpp — header-only helper (unchanged from v1.1):
- miniapps/seas/common/device_init.hpp — header-only helper:

```
namespace mfem { namespace seas {
```

```

// Initialise mfem::Device given a config string ("cuda", "hip", "cpu", or
// "cpu-debug"). Must be called AFTER Mpi::Init() and AFTER Hypre::Init()
// but BEFORE any Vector/HypreParMatrix/HypreSolver construction (because
// Device::Configure triggers Hypre::InitDevice when MPI is enabled).
//
// On root rank only, prints device info via Device::Print().
// Throws (MFEM_VERIFY) on a config that names a backend not compiled in.
void InitDevice(const std::string &config, bool print_on_root = true,
               MPI_Comm comm = MPI_COMM_WORLD);

// Returns "cuda" / "hip" / "cpu" – the resolved backend after
// Device::Configure. Useful for downstream branches.
std::string GetActiveBackend();

}} // namespace mfem::seas

```

- miniapps/seas/tests/unit/test_device_init.cpp — unit test:
 - TEST(DeviceInit, CpuConfigDoesNotEnableDeviceBackend) — InitDevice("cpu") then assert !Device::Allows(Backend::CUDA_MASK | Backend::HIP_MASK).
 - TEST(DeviceInit, CudaConfigEnablesDevice) — gated on #ifdef MFEM_USE_CUDA; asserts Device::Allows(Backend::CUDA).

Files to Modify

- miniapps/seas/Makefile — add a MFEM_USE_CUDA ?= and MFEM_USE_HIP ?= detection block reading the value from \$(CONFIG_MK). Emit a clear error if MFEM_USE_CUDA=YES and HYPRE was built without GPU support (detect by grepping HYPRE_OPT for HYPRE_USING_GPU). Phase 0 introduces no kernel files (no .cu / .hip.cpp), so the Makefile change is limited to (a) reading the new flags, (b) emitting build-time compatibility checks, and (c) adding an empty placeholder SEAS_KERNEL_OBJS := variable that Phase 3 will populate with the first kernel objects. The nvcc / hipcc pattern rules are added in Phase 3 (not Phase 0). [R-GPU-015] The same Makefile must also detect when both MFEM_USE_CUDA=N0 and MFEM_USE_HIP=N0 (Aurora CPU fallback) and skip the GPU compat check entirely. [V2]
- miniapps/seas/CMakeLists.txt — mirror the Makefile flag plumbing for the optional CMake build.
- miniapps/seas/drivers/seas_driver.cpp — at the top of main (after MPIContext mpi(&argc, &argv) which already calls Mpi::Init + Hypre::Init per common/mpi_context.hpp:36–47):

```

std::string device_config = "cpu"; // default
// parse --device <config> from argv after the positional args
for (int i = 2; i < argc; i++) {
    std::string a(argv[i]);
    if (a == "--device" && i + 1 < argc) { device_config = argv[++i]; }
}
// Also accept [runtime].device from TOML (parsed at line 227);
// CLI overrides TOML.
if (!cli_device_set && !config.runtime.device.empty()) {
    device_config = config.runtime.device;
}
mfem::seas::InitDevice(device_config, mpi.IsRoot(), mpi.GetComm());

```

- miniapps/seas/drivers/tpv102_driver.cpp (line 413), tpv104_driver.cpp (line 412), tpv205_driver.cpp (line 452) — these drivers use raw MPI_Init(&argc, &argv) (NOT MPIContext) and do NOT call Hypre::Init because they are explicit (no linear solve). Insert seas::InitDevice directly after the MPI_Init / MPI_Comm_rank / MPI_Comm_size block (at lines 415–423 of tpv102_driver.cpp, equivalent positions in the other two). Do NOT add an MPI-Context construction here — it would pull Hypre::Init into an explicit-solver build for no reason.

seas::InitDevice still works in this configuration because Device::Configure gates the Hypre-init step on HYPRE_Initialized() (general/device.cpp:287). Example for tpv102_driver.cpp:

```
#ifdef MFEM_USE_MPI
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    int rank, nprocs;
    MPI_Comm_rank(comm, &rank);
    MPI_Comm_size(comm, &nprocs);
#else
    int rank = 0, nprocs = 1;
#endif

    // R-GPU-002: device init for explicit drivers – no Hypre::Init here.
    std::string device_config = GetStringArg(argc, argv, "--device", "cpu");
    mfem::seas::InitDevice(device_config, rank == 0, comm);
```

[R-GPU-002]

- miniapps/seas/config/seas_config.hpp — add a RuntimeConfig struct to SEASConfig:

```
struct RuntimeConfig {
    std::string device = ""; // "", "cpu", "cuda", "hip", "debug"
};
struct SEASConfig {
    // ... existing fields ...
    RuntimeConfig runtime;
};
```

- miniapps/seas/config/seas_config_parser.hpp — parse the new TOML block [runtime] (default empty / cpu):

```
[runtime]
device = "cuda" # optional; "cpu" is the default
```

- miniapps/seas/config/seas_config_parser.hpp::Validate — extend to validate that runtime.device ∈ {"", "cpu", "cuda", "hip", "debug"}. Single-backend strings only; multi-backend strings ("cuda,debug" etc.) are parsed by Device::Configure but their interaction with HYPRE / PETSc is undertested — defer to Phase 7. [R-GPU-011]

Detailed Requirements

1. Initialization order (HARD CONTRACT, do not deviate): The order depends on the driver:

- BP5 driver (**seas_driver.cpp**) — uses MPIContext which already calls Mpi::Init + Hypre::Init in its constructor (common/mpi_context.hpp:36–47). Insert seas::InitDevice after the MPIContext mpi(&argc, &argv) line. Effective order: Mpi::Init() → Hypre::Init() → Device::Configure(...).
- TPV102/104/205 drivers — use raw MPI_Init; do NOT call Hypre::Init (no linear solve). Insert seas::InitDevice directly after the MPI_Init / MPI_Comm_rank block. Effective order: MPI_Init() → Device::Configure(...). Hypre is never initialized; Device::Configure gates its Hypre::InitDevice call on HYPRE_Initialized() so this is safe (general/device.cpp:287).

Device::Configure internally re-invokes Hypre::InitDevice only if Hypre is already initialized — see linalg/hypre.hpp:88–102. No HypreParMatrix, HypreParVector, or HypreSolver may be constructed before InitDevice returns (BP5 only — irrelevant for the explicit TPV drivers). Phase 0 audit: grep HypreParMatrix\|HypreParVector\|HypreSolver in seas_driver.cpp; confirm none appear above the InitDevice call. [R-GPU-002]

2. Function signature for `seas::InitDevice`:

```
void InitDevice(const std::string &config,
               bool print_on_root,
               MPI_Comm comm)
{
    // Validate the string. Accept: "cpu", "cuda", "hip", "debug"
    // (single backends only). Empty string → "cpu".
    const std::string cfg = config.empty() ? "cpu" : config;

    // Idempotency guard (R-GPU-006): MFEM's Device::Configure is
    // additive (MarkBackend uses |=) – calling twice with different
    // strings ORs both backends into the active set without warning.
    // Guard with a function-static "first call" flag plus a cached
    // config to make repeated calls a no-op on match and a warning on
    // mismatch. Tests that explicitly want to reset the device must
    // bypass this helper.
    static std::string s_active_cfg;
    static bool s_initialized = false;
    if (s_initialized)
    {
        if (s_active_cfg != cfg && print_on_root)
        {
            mfem::out << "seas::InitDevice: already configured with '"
                       << s_active_cfg << "', ignoring re-init request for '"
                       << cfg << "'." << std::endl;
        }
        return;
    }

    // The MFEM Device::Configure call accepts MFEM's backend syntax
    // ("cuda", "cuda:debug", "raja-cuda", "hip", "occa-cuda", etc.).
    // We pass it through verbatim and let MFEM emit the diagnostic
    // if a backend is missing.
    mfem::Device::Configure(cfg.c_str());
    s_active_cfg = cfg;
    s_initialized = true;

#ifdef MFEM_USE_MPI
    int rank;
    MPI_Comm_rank(comm, &rank);
    if (print_on_root && rank == 0) { mfem::Device::Print(); }
#else
    if (print_on_root) { mfem::Device::Print(); }
#endif
}
```

3. CLI parsing rules (each driver):

- `--device cuda` → `device_config = "cuda"`.
- `--device hip` → `device_config = "hip"`.
- `--device cpu` → `device_config = "cpu"` (explicit; same as default).
- `--device debug` → `device_config = "debug"` — MFEM's debug device that simulates device behavior on host (catches missing `Read()`/`Write()`).
- Unknown value → `MFEM_ABORT("unknown --device value: " << config)`.
- CLI flag overrides `[runtime].device` from TOML.

4. TOML parsing rule (`seas_config_parser.hpp`):

- Read `[runtime].device` if present; default empty.
 - In `Validate`, reject any value not in the single-backend set `{ "", "cpu", "cuda", "hip", "debug" }`. Multi-backend strings (e.g., `"cuda,debug"`) are valid for `Device::Configure` but the interaction with HYPRE / PETSc is understested; defer to Phase 7. [R-GPU-011]
5. Smoke test acceptance procedure:
- Build CUDA binary, run TPV102 1-second sample with `--device cpu`, diff against a pre-change `tpv102_cpu.txt` baseline — must be bit-identical.
 - Same for `seas_driver bp5_500m.toml --device cpu --max-steps 10`.

Interfaces

- New: `mfem::seas::InitDevice(...)` (per signature above).
- New: `mfem::seas::GetActiveBackend()` returning `"cuda"|"hip"|"cpu"`.
- New: TOML `[runtime].device` field.
- New: `--device <cfg>` CLI flag on all four drivers.

Edge Cases to Handle

- Repeated **`Device::Configure`** calls — MFEM does NOT enforce single-call semantics; `Device::Configure` is additive — a second call with a different backend ORs that backend into the active set (see `general/device.cpp:242`, `MarkBackend(b) { backends |= b; }`). The `seas::InitDevice` function-static guard `cached_cfg + initialized` turns the second call into a no-op when the config matches, or emits a warning (root rank only) when it differs. Do not attempt to “abort on conflicting configs” — there is no reliable cross-call comparison once the bitwise OR has happened. [R-GPU-006]
- **`--device cuda`** on a CPU-only build — MFEM’s `Device::Configure` will abort with a clear message. Acceptance: that abort happens cleanly with the build-time configuration listed.
- MPI + CUDA without CUDA-aware MPI — Phase 0 does not depend on CUDA-aware MPI; CPU-side ghost exchange in subsequent phases will trigger device↔host transfers. Document this fact in `gpu_dev/CUDA_AWARE_MPI_NOTE.md` (created Phase 4).
- PETSc-built-without-CUDA + **`--device cuda`** — if `config.time.use_petsc_ts == true` and `device_config != "cpu"` but `!PetscDeviceContextGetDevice(...)`, abort with a clear error in `seas_driver.cpp` before time stepping begins.

Acceptance Criteria

- ☐ `build_polaris.sh` builds MFEM + SEAS on a Polaris compute node (or login node with gpu queue access); produces a binary that runs `tpv102_driver --device cuda --tfinal 0.01` to completion on 1 A100. [V2]
- ☐ `build_frontier_olcf.sh` builds MFEM + SEAS on Frontier; produces a binary that runs `tpv102_driver --device hip --tfinal 0.01` to completion on 1 MI250X GCD. [V2]
- ☐ `build_summit.sh` builds MFEM + SEAS on Summit; produces a binary that runs `tpv102_driver --device cuda --tfinal 0.01` to completion on 1 V100 (NOTE the CC-7.0 warning emitted by the script). [V2]
- ☐ `build_aurora.sh` builds MFEM + SEAS on Aurora in CPU-only mode; produces a binary that runs `tpv102_driver --device cpu --tfinal 0.01` to completion on 1 Xeon Max socket (PVC GPUs idle pending Aurora gap resolution). [V2]
- ☐ All four drivers accept `--device cpu|cuda|hip|debug` and abort cleanly on unknown values.
- ☐ `test_device_init` passes on CPU build and CUDA build (and HIP build, on a Frontier login node or local AMD test box).
- ☐ TPV102 1-s smoke run with `--device cpu` is byte-identical to pre-change reference output on EACH platform (Polaris, Frontier, Summit, Aurora) — uses the same CPU code path on all. [V2]
- ☐ BP5 500m 10-step smoke run with `--device cpu` is byte-identical to pre-change reference output on each platform.

Dependencies

- Depends on: nothing.
 - Required by: Phases 1–6.
-

Phase 1: BP5 sparse solve on GPU via HYPRE

Goal

After Phase 1: the BP5 quasi-dynamic linear solve $Ku = b$ runs on the GPU through HypreBoomerAMG + HyprePCG when `--device cuda|hip` is selected, with no kernel writing on the SEAS side (HYPRE provides the device kernels).

Files to Create

- `miniapps/seas/tests/unit/test_bp5_hyre_gpu.cpp` — CPU↔GPU equivalence test for the assembled K solve at BP5 500m mesh on a fixed RHS.

Files to Modify

- `miniapps/seas/domain/elasticity_operator_assembly.inl`:
 - **SetupSolver()** / inside **AssembleStiffness()**: when `mfem::Device::Allows(Backend::DEVICE_MASK)` is true, reject `SolverType::MUMPS`, `SolverType::SUPERLU`, `SolverType::STRUMPACK` with `MFEM_ABORT("Solver type X has no GPU path; choose CG_AMG or \"run with --device cpu\")`. Reason: the project's CLAUDE.md forbids silent fallbacks.
 - When the device backend is active and `solver_type == CG_AMG`, set `cg->iterative_mode = false`, ensure preconditioner is HypreBoomerAMG, and call `cg->SetOperator(*K)` — these already happen on lines 388–400, so the change is a pre-condition check only.
- `miniapps/seas/domain/elasticity_operator.hpp`:
 - Add accessor `bool RequiresHostSolver()` const that returns true for MUMPS/SuperLU/STRUMPACK and false for CG_AMG/GMRES_AMG.
- `miniapps/seas/config/seas_config_bridge.hpp`:
 - In `ParseSolverType`, add a warning (root rank) when both `runtime.device != "cpu"` and `solver_type ∈ {mumps, superlu, strumpack}` are set in the same TOML, naming the file and line.
- `miniapps/seas/drivers/seas_driver.cpp`:
 - After `seas::InitDevice` call and before `ElasticityDomainOperator domain(...)` construction (around line 327), add:

```
if (mfem::Device::Allows(mfem::Backend::DEVICE_MASK) &&
    (solver_type == SolverType::MUMPS ||
     solver_type == SolverType::SUPERLU ||
     solver_type == SolverType::STRUMPACK)) {
    if (mpi.IsRoot()) {
        std::cerr << "ERROR: solver_type='" << config.solver.solver_type
                    << "' has no GPU path. Use solver_type=\"cg\" \"
                    << "or run with --device cpu.\n";
    }
    MFEM_ABORT("Incompatible solver_type for GPU run");
}
```
- `miniapps/seas/bp5/parametric_study/case_500m/bp5_500m.toml`:
 - Add commented hint: `# For GPU runs, set solver_type = "cg".`

Detailed Requirements

1. No SEAS-side kernel changes. HYPRE 2.31.0+ automatically runs BoomerAMG and PCG on whichever device `Device::Configure` selected, provided HYPRE was built with `--with-cuda / --with-hip` and `Hypre::configure_runtime_policy_from_mfem == true` (default per `linalg/hypre.hpp:119`).
2. Memory residency — `cached_Ah_` (`HypreParMatrix`), `B_`, `X_` (`HypreParVectors`) automatically live in device memory when the device backend is active because their underlying memory class is `GetHypreMemoryClass()` (`linalg/hypre.hpp:177–200`). No `UseDevice(true)` call needed on these — they're managed.
3. **Vector** interop — the RHS and solution Vectors passed into `solver_>Mult(B_, X_)` are MFEM Vectors, not `HypreParVectors`. The `ParallelAssemble / Distribute` calls inside the existing assembly code already convert between the two. Audit those call sites and add `UseDevice(true)` on any Vector that funnels into / out of the linear solve:
 - `elasticity_operator_assembly.inl:215–409` (`cached_Ah_` assembly).
 - `elasticity_operator_traction.inl` for the displacement view.
 - `elasticity_operator_setup.inl` for RHS `b_`.
4. RHS rebuild stays on CPU in Phase 1. The RHS is rebuilt every ODE-RHS call by `AssembleSlipContribution` and `AssembleDirichletLoading` (`elasticity_operator_assembly.inl:420` and `1040`) — both still run on CPU in Phase 1. After build, the assembled RHS is copied to a `HypreParVector` whose memory class triggers a host→device transfer. This is acceptable for Phase 1 (RHS is small relative to solve cost); Phase 6 ports the RHS construction.
5. AMG configuration on GPU — the GPU path inside HYPRE prefers `l1Jacobi` smoother over `l1GS` per `linalg/hypre.cpp:1504,1840,1853,2567,...`. No SEAS-side configuration needed: `HypreBoomerAMG`'s default already picks the right smoother via `HypreUsingGPU()`.

Interfaces

- New: `ElasticityDomainOperator::RequiresHostSolver()` `const`.
- Modified contract: if `Device::Allows(DEVICE_MASK)` is true, `SolverType::MUMPS, SUPERLU, STRUMPACK` causes `MFEM_ABORT` at construction.

Edge Cases to Handle

- `--device cuda + solver_type=mumps` in TOML — abort with named file (TOML path), named alternative (cg), and exit code 1.
- HYPRE compiled without CUDA, `--device cuda` — `Hypre::InitDevice` is a no-op per its docstring; HYPRE solvers stay on CPU even though MFEM thinks the device is active. Detection:
 - Inspect `GetHypreMemoryClass()` after `seas::InitDevice` returns.
 - If `Device::Allows(DEVICE_MASK)` is true but `GetHypreMemoryClass() == MemoryClass::HOST`, emit a warning (root only) and continue (HYPRE will silently run on CPU; the SEAS-written kernels in Phases 3–5 still run on device).
- MUMPS deprecation propagation — `bp5_500m.toml` defaults to MUMPS. Phase 1 does NOT change the default; only enforces the gate. A follow-up commit (post-Phase 1) can switch the default to "cg" if the GPU runs prove faster.

Acceptance Criteria

- ☐ `test_bp5_hypre_gpu` passes: same K, same b, GPU and CPU BoomerAMG-preconditioned PCG both converge within $2\times$ iteration count of each other, with L2 relative diff $< 1e-6$ at PCG tolerance $1e-8$. (Loose vs. originally $1e-10$ because the GPU AMG uses `l1Jacobi` smoother where CPU uses `l1GS` — see `linalg/hypre.cpp:1504,1840,...`) [R-GPU-007]
- ☐ BP5 500m run with `--device cuda` and `solver_type="cg"` completes 100 steps without abort.
- ☐ BP5 500m run with `--device cuda` and `solver_type="mumps"` aborts with the new error message at startup.
- ☐ `chunhui-benchmark` PASS on BP5 100s short run (CPU cg+amg vs GPU cg+amg comparison within existing 5% envelope).

Dependencies

- Depends on: Phase 0.
 - Required by: nothing else strictly, but Phase 6 builds on the device-resident memory pattern established here.
-

Phase 2: Friction-solver kernels on GPU (BP5 + dynamic rupture)

Goal

After Phase 2: the per-DOF / per-fault-QP friction root-find loop runs on the device with byte-equivalent (to NR tolerance) results to the CPU Brent path.

Files to Create

- miniapps/seas/dynamic/kernels/friction_kernels.hpp — header-only, inline-able device-compatible NR routines. Pattern:

```
namespace mfem { namespace seas { namespace kernels {  
  
  // Stable Newton-Raphson rate-and-state slip-rate solve, matching  
  // FrictionSolver::SolveNRStable but device-friendly.  
  //  
  // Input: tau, psi, sigma_n, eta, a, V_prev_guess. Output: V.  
  // Fixed 15 iterations + early-exit on |g| < 1e-12.  
  //  
  // MUST compile under nvcc / hipcc with no host-only dependencies.  
  MFEM_HOST_DEVICE inline  
  real_t SolveSlipRateNRStableDevice(real_t tau, real_t psi, real_t sigma_n,  
                                     real_t eta, real_t a, real_t V_init);  
  
  // Vector-valued version: anti-parallel decomposition of slip rate from  
  // total traction vector (tau_dip, tau_strike). Matches  
  // DieterichRuinaFriction::SolveSlipRateVectorPsi.  
  MFEM_HOST_DEVICE inline  
  void SolveSlipRateVectorNRDevice(const real_t tau_vec[2], real_t psi,  
                                   real_t sigma_n, real_t eta, real_t a,  
                                   real_t V_vec[2], real_t *V_abs_out);  
  
}}} // namespaces
```

- miniapps/seas/tests/unit/test_friction_device.cpp — CPU/GPU equivalence over a 64-point parameter sweep matching the existing test_friction_law.cpp cases.

Files to Modify

- miniapps/seas/dynamic/friction_solver.hpp:
 - Make SolveSlipRateNewtonStable's computational core call kernels::SolveSlipRateNRStableDevice so the same routine drives CPU and device paths (single source of truth).
- miniapps/seas/fault/rate_state_fault.hpp:
 - **ComputeRHS** (line 354) — convert the host for (int i = 0; i < num_nodes_; i++) loop into a device kernel when Device::Allows(DEVICE_MASK):

```
if (mfem::Device::Allows(mfem::Backend::DEVICE_MASK)) {  
  ComputeRHSDevice(traction, state, rate, normal_traction);  
}
```

```

} else {
    ComputeRHSHost(traction, state, rate, normal_traction); // existing body
}

```

- Add void ComputeRHSDevice(...) implementation that:
 - Calls traction.Read(), state.Read(), rate.Write(), normal_traction->Read(), tau_pre_.Read(), geom_->GetAValues().Read(), geom_->GetEtaValues().Read(), geom_->GetDepths().Read().
 - At first call, reads the runtime-known evolution-law identity from evolution_->GetName() and caches an evolution_kind_ enum value (EvolutionKind::AgingPsi, SlipPsi, Aging, Slip). This is necessary because StateEvolution::Rate is a pure virtual function (friction/state_evolution.hpp:41) and virtual dispatch through a host-side vtable is illegal in a device kernel. Tagging the virtual method MFEM_HOST_DEVICE does NOT make device dispatch work — the implementer must NOT call evolution_->Rate(...) from inside mfem::forall.
 - Launches mfem::forall(num_nodes_, ...) with the rate formulas coded as MFEM_HOST_DEVICE free functions in a new file friction/state_evolution_kernels.hpp (one free function per law identity) and selected by a switch on the cached evolution_kind_ enum inside the kernel body.
 - Tracks V_max_ via a device-side reduction (Pattern A in §“Detailed Requirements” — scratch v_abs_scratch_ Vector with UseDevice(true) + post-kernel host call to v_abs_scratch_.Max(), which runs on the device).
- Add private member EvolutionKind evolution_kind_ and populate it in the constructor from evolution_->GetName().
- Mark tau_pre_, slip_rate_ with tau_pre_.UseDevice(true) and slip_rate_.UseDevice(true) in the Init step.
- miniapps/seas/friction/state_evolution_kernels.hpp (NEW) — declares:

```

namespace mfem { namespace seas { namespace kernels {

enum class EvolutionKind : int { Aging = 0, Slip = 1,
                                AgingPsi = 2, SlipPsi = 3 };

// One MFEM_HOST_DEVICE free function per law identity. Each one
// reproduces the body of the corresponding StateEvolution subclass's
// Rate(V, theta_or_psi, Dc) method bit-for-bit, with the runtime
// material parameters (b, V0, f0) passed as scalar arguments.

MFEM_HOST_DEVICE inline
real_t AgingLawPsiRateDevice(real_t V, real_t psi, real_t Dc,
                             real_t b, real_t V0, real_t f0);

MFEM_HOST_DEVICE inline
real_t SlipLawPsiRateDevice(real_t V, real_t psi, real_t Dc,
                             real_t b, real_t V0, real_t f0);

MFEM_HOST_DEVICE inline
real_t AgingLawRateDevice(real_t V, real_t theta, real_t Dc);

MFEM_HOST_DEVICE inline
real_t SlipLawRateDevice(real_t V, real_t theta, real_t Dc);

}}} // namespaces

```

Bodies must be 1:1 ports of the corresponding host class methods in friction/state_evolution.hpp. Do not modify the existing virtual classes' Rate methods (the host CPU path keeps calling the virtuals).

- miniapps/seas/fault/fault_geometry.hpp:

- Mark `a_values_`, `eta_values_`, `depths_`, `dc_values_`, `coords_x2_`, `coords_x3_` all `UseDevice(true)` after population.
- `miniapps/seas/friction/state_evolution.hpp`:
 - Do NOT tag the virtual Rate method `MFEM_HOST_DEVICE` — that has no effect on device dispatch and is misleading. The virtuals stay host-only; device kernels use the free functions in `state_evolution_kernels.hpp` (new file above).
- `miniapps/seas/friction/dieterich_ruina.hpp`:
 - Tag `FrictionCoefficientPsi`, `Asinh`, `residual` / derivative helpers with `MFEM_HOST_DEVICE` (these are non-virtual free functions / static methods — device-tagging works). [R-GPU-001]

Detailed Requirements

1. Single source of truth for the NR formula. Both CPU `FrictionSolver::SolveNRStable` and device `kernels::SolveSlipRateNRStableDevice` must call the same `MFEM_HOST_DEVICE` core routine. Pre-condition: `friction_coeff_stable.hpp::FrictionCoefficientStable` is already `MFEM_HOST_DEVICE`-compatible (audit: no `std::vector`, no virtual).
2. Initial guess. `V_init` for NR must come from `slip_rate_(i)` of the previous step (warm start) — this matches `FrictionSolver::SolveNRStable`'s warm-start contract.
3. Convergence policy. Fixed 15 iterations + early-exit on $|g| < 1e-12 * (|\sigma_n * a| + |\eta * V| + |\tau|)$ (relative tolerance on the residual scale). Caller passes `V_init`; on non-convergence, write `V = -1.0` as a sentinel and atomically increment a global `non_converged_count_dev` counter checked post-kernel; abort if non-zero.
4. `V_max` reduction. Only one viable pattern:
 - A. Per-DOF kernel writes `V_abs` to a scratch `Vector v_abs_scratch_(num_nodes_)`, marked `UseDevice(true)`. Post-kernel host call to `v_abs_scratch_.Max()` — `MFEM Vector::Max()` runs on the device when `UseDevice(true)` (`linalg/vector.cpp:1200` uses `mfem::reduce` internally).
 - B. Atomic-max inside the kernel is NOT VIABLE — MFEM provides only `mfem::AtomicAdd<T>` in `general/backends.hpp:94`; there is no `AtomicMax` template and CUDA's `atomicMax` does not have a double overload. Do not pursue this path. Use A (the only available path). [R-GPU-004]
5. Below-fault hardcoded branch (BP1/BP2 antiplane only, **`SlipComponents == 1`**): the device kernel must reproduce the host branch at lines 380–388 of `rate_state_fault.hpp` — uniform rate = `params_.Vp` for DOFs with `depths(i) < -params_.Wf`. Encode as a per-DOF conditional inside the kernel body.

Interfaces

- New: `mfem::seas::kernels::SolveSlipRateNRStableDevice(...)` (`MFEM_HOST_DEVICE`).
- New: `mfem::seas::kernels::SolveSlipRateVectorNRDevice(...)` (`MFEM_HOST_DEVICE`).
- New: `RateStateFaultOperator::ComputeRHSDevice(...)` (private).
- No public-API breakage; the `ComputeRHS` dispatch is transparent.

Edge Cases to Handle

- **`SlipComponents == 1`** (antiplane / BP1 / BP2) vs **`== 2`** (3D BP5): template specialisation; both must have device kernels.
- **`normal_traction == nullptr`** (BP1/BP2 path): the device kernel takes a `const real_t *normal_traction_data = nullptr` and branches on it.
- Brent-vs-NR drift — Brent (CPU) and NR (GPU) can produce slip rates differing by $\sim 1e-8$ in pathological near-zero-traction regions. The CPU/GPU equivalence test uses `NewtonRaphsonStable` on both sides for apples-to-apples; the Brent-vs-NR drift is captured by the existing `test_friction_law.cpp`.
- Warm-start **`V_init = 0`** — first call from `Init` has no previous slip rate. Use `V_init = 1e-12` (or `params_.V_init`) to keep NR Jacobian non-singular.

Acceptance Criteria

- `test_friction_device` passes: device NR matches host NR within $1e-14$ (same formula) on the 64-point sweep.
- `test_friction_law` (existing) still passes (NR vs Brent at $1e-8$ tolerance).
- BP5 100-step run with `--device cuda` reproduces CPU run within $1e-10$ on `slip_rate` time series at all probe stations.
- TPV102 1-s run with `--device cuda` (friction-only-on-GPU; volume still on CPU via copy-back) produces fault `V_max` time series within $1e-10$ of CPU run.

Dependencies

- Depends on: Phase 0 (`Device::Configure`).
 - Required by: Phase 4 (face flux fault branch reuses these device friction routines).
-

Phase 3: Wave-operator element-local kernels (volume + mass inverse + spatial derivative)

Goal

After Phase 3: the three element-local hot routines of `WaveOperator` (`ComputeVolumeRHS`, `ApplyMassInverse`, `ApplySpatialDerivative`) run on the device. Mult still falls back to host for `ComputeFaceFluxRHS` and `ComputeSharedFaceFluxRHS`; an extra copy-back is inserted between volume and face steps and tagged with a TODO removed by Phase 4.

Files to Create

- `miniapps/seas/dynamic/kernels/wave_volume_kernels.hpp` — declares:

```
namespace mfem { namespace seas { namespace kernels {

// Compute volume RHS for the 3D velocity-stress system on tets.
//
// Inputs:
//   Q          size NUM_STATE * ndof_total
//   B, G, w, detJ  per-element shape / dshape / weight / det(J) tables,
//                  precomputed via mesh.GetGeometricFactors + DofToQuad.
//   Ax, Ay, Az    9x9 Jacobian matrices, constant across elements.
//   ne, ndof, nqp geometry counts.
//
// Output:
//   rhs          additive accumulation into rhs[c * ndof_total + e*ndof + i]
//
// Reshape conventions documented inline; matches the layout MFEM's
// DofToQuad NATIVE ordering on a tet of degree p produces.
void ComputeVolumeRHSDevice(const Vector &Q,
                           const QuadratureData &qd, // see common_kernels.hpp
                           const DenseMatrix &Ax,
                           const DenseMatrix &Ay,
                           const DenseMatrix &Az,
                           Vector &rhs);

// Per-element symmetric SPD mass-matrix inverse application (9 RHS).
// elem_mass_inv: ne * ndof * ndof (packed row-major in a Vector).
void ApplyMassInverseDevice(const Vector &elem_mass_inv,
```

```

        int ne, int ndof,
        Vector &dQdt);

// Element-local L2-projected spatial derivative in direction `dir`.
// Output: dQ_dxdir same size as Q. Uses cached K_d^e and M_e^{-1}.
void ApplySpatialDerivativeDevice(int dir, const Vector &Q,
    const Vector &Kd_packed,
    const Vector &Minv_packed,
    int ne, int ndof,
    Vector &dQ_dxdir);

}}} // namespaces

```

- miniapps/seas/dynamic/kernels/common_kernels.hpp — shared device utilities:

```

struct QuadratureData {
    // Precomputed once at WaveOperator construction; lives on device.
    Vector B;          // shape values at qp:  ndof * nqp
    Vector G;          // physical dshape at qp: ndof * 3 * nqp * ne
    Vector w_detJ;     // ip.weight * det(J):  nqp * ne
};

void BuildQuadratureData(const FiniteElementSpace &fes, int order,
    QuadratureData &out);

```

- miniapps/seas/tests/unit/test_wave_volume_device.cpp — CPU/GPU equivalence over the existing TPV102 200m mesh on a fixed Q.
- miniapps/seas/tests/unit/test_wave_mass_inverse_device.cpp — ditto for mass inverse.
- miniapps/seas/tests/unit/test_ader_spatial_derivative_device.cpp — ditto for ApplySpatialDerivative on directions 0, 1, 2.

Files to Modify

- miniapps/seas/dynamic/wave_operator.hpp:

– Add private members:

```

mutable kernels::QuadratureData qd_;
mutable Vector elem_mass_inv_packed_; // size ne_ * ndof_per_el_^2
mutable Vector elem_Kd_packed_[3];    // K_d per direction
bool device_buffers_initialized_ = false;
void InitDeviceBuffers() const;        // lazy init at first device call

```

– Add accessor bool DeviceIsActive() const { return mfem::Device::Allows(mfem::Backend::DEVICE); }.

- miniapps/seas/dynamic/wave_operator.inl:

– **ComputeVolumeRHS** (line 772): branch on DeviceIsActive():

* true → ensure InitDeviceBuffers() was called, then dispatch kernels::ComputeVolumeRHSDevice(Q, qd_, Ax_, Ay_, Az_, rhs).
 * false → existing host body unchanged.

– **ApplyMassInverse** (line 4897): same branch; device dispatch kernels::ApplyMassInverseDevice(elem_mass_inv_packed_, ne_, ndof_per_el_, dQdt).

– **ApplySpatialDerivative** (line 847): branch; device dispatch kernels::ApplySpatialDerivativeDevice(Q, elem_Kd_packed_[dir], elem_mass_inv_packed_, ne_, ndof_per_el_, dQ_dxdir).

– **AssembleElementMassInverse** (line 4928): leave host computation in place; at the end, also pack the inverses into elem_mass_inv_packed_ as elem_mass_inv_packed_[e*ndof*ndof + i*ndof + j] = elem_mass_inv_[e](i,j) and call elem_mass_inv_packed_.UseDevice(true).

– **Mult** (line 684):

- * Insert `if (DeviceIsActive()) { dQdt.UseDevice(true); }` at the top.
- * Between `ComputeFaceFluxRHS(host)` and `ApplyMassInverse(device)`: add `dQdt.HostReadWrite()` to ensure face flux contributions are flushed to device memory before mass-inverse runs.
- * Add a `// TODO[G-301]: remove HostReadWrite after Phase 4 lands face flux on device.` comment.
- `miniapps/seas/drivers/tpv102_driver.cpp`, `miniapps/seas/drivers/tpv104_driver.cpp`, `miniapps/seas/drivers/tpv205_driver.cpp`:
 - DRIVER CONTRACT (R-GPU-014): the ODE state `Q` (the Vector passed into `ode_solver->Init(...)`) MUST be `UseDevice(true)` BEFORE the time loop starts, so per-step `Mult` does not trigger host→device copies of the full state. Add immediately after the state vector is allocated and before `ode_solver->Init(...)`:


```
if (mfem::Device::Allows(mfem::Backend::DEVICE_MASK)) {
    Q.UseDevice(true);
    // Any other long-lived per-step Vector that flows through
    // Mult – e.g., the RK scratch buffers if they are owned at
    // driver scope – should also be flipped here.
}
```
 - The same flip applies in `drivers/seas_driver.cpp` for the BP5 state vector before `PetscODES-olver/DormandPrinceRK45::Init`.

Detailed Requirements

1. Layout (CRITICAL). Existing `Q` storage is component-major: `Q[c * ndof_total_ + e * ndof_per_el_ + i]`. Device kernels MUST preserve this. Document with a `static_assert` and a comment at the top of `wave_volume_kernels.hpp`.
2. **QuadratureData** packing. Built once at the first call to `InitDeviceBuffers()`:
 - `B[i * nqp + q] = shape_i(x_q)` — independent of element for reference-element basis on uniform tets; pack per typical FE element, asserting all elements have identical geometry type and order. If heterogeneous, abort.
 - `G[((e * ndof + i) * 3 + d) * nqp + q]` — physical-space $d\text{shape } \partial\phi_i/\partial x_d$ at QP `q` on element `e`.
 - `w_detJ[e * nqp + q] = ip.weight * Tr.Weight()`.
 - Mark all three `UseDevice(true)`.
3. Volume kernel structure. The HOST SETUP block below is part of the contract — implementer must NOT use raw `Q.GetData()` (host-only pointer) inside any kernel. All inputs must come through `Vector::Read()` / `Array::Read()` and all outputs through `ReadWrite()` / `Write()`. [R-GPU-008]

```
// HOST SETUP (run before mfem::forall – captures device pointers
// and creates DeviceTensor reshape views):
const real_t *Q_data = Q.Read(); // device pointer; UseDevice
real_t *rhs_data = rhs.ReadWrite(); // additive into existing
const auto B = Reshape(qd.B.Read(), ndof, nqp);
const auto Jinv = Reshape(qd.Jinv.Read(), 3, 3, ne); // per-elem
const auto dsh = Reshape(qd.dshape_ref.Read(), ndof, 3, nqp);
const auto wd = Reshape(qd.w_detJ.Read(), nqp, ne);
const auto Ax_d = Reshape(Ax_packed.Read(), NUM_STATE, NUM_STATE);
const auto Ay_d = Reshape(Ay_packed.Read(), NUM_STATE, NUM_STATE);
const auto Az_d = Reshape(Az_packed.Read(), NUM_STATE, NUM_STATE);
const int ndof_total_capture = ndof_total; // capture by value

mfem::forall(ne, [=] MFEM_HOST_DEVICE (int e) {
    real_t Q_qp[NUM_STATE]; // 9 doubles per QP – fits in
    // registers on A100/MI250X

    for (int q = 0; q < nqp; q++) {
        // 1. Evaluate Q at QP (use captured Q_data pointer from Read()).
```

```

    for (int c = 0; c < NUM_STATE; c++) {
        Q_qp[c] = 0.0;
        for (int i = 0; i < ndof; i++) {
            Q_qp[c] += B(i, q)
                * Q_data[c*ndof_total_capture + e*ndof + i];
        }
    }
    // 2. Compute  $F = \sum_d A_d * Q_{qp}$  (3 9x9 matvecs).
    // 3. Build physical-space dshape on the fly:  $G(e,i,d,q) =$ 
    //      $\sum_m J_{inv}(d,m,e) * dsh(i,m,q)$ .
    // 4. Accumulate  $(\nabla\phi_i, F_d)$ .
    const real_t w = wd(q, e);
    for (int c = 0; c < NUM_STATE; c++) {
        for (int i = 0; i < ndof; i++) {
            real_t val = 0.0;
            for (int d = 0; d < 3; d++) {
                // val += G(e, i, d, q) * F[d][c] computed inline.
            }
            // No atomics – each (e, c, i) is written by exactly one
            // thread (the one handling e).
            rhs_data[c*ndof_total_capture + e*ndof + i] += w * val;
        }
    }
}
});

```

- No race: each (e, c, i) triple is written by exactly one thread (the one handling element e).
- For larger ndof, consider 2-level parallelism (MFEM_FORALL_2D(e, ne, ndof, ...)) — defer to Phase 7 tuning.
- Implementer rule: apply the same HOST SETUP pattern to the mass-inverse and spatial-derivative kernels below — every input Vector capture goes through .Read(), every output through .ReadWrite() or .Write().

4. Mass-inverse kernel structure.

```

mfem::forall(ne, [=] MFEM_HOST_DEVICE (int e) {
    const real_t *Minv_e = Minv_data + e * ndof * ndof;
    real_t rhs_local[MAX_DOF_TET]; // small-cap stack array; MAX_DOF_TET=35 for p=4 tet
    real_t res_local[MAX_DOF_TET]; // loop bounds use runtime ndof, not MAX_DOF_TET
    for (int c = 0; c < NUM_STATE; c++) {
        for (int i = 0; i < ndof; i++) {
            rhs_local[i] = dQdt[c*ndof_total + e*ndof + i];
        }
        for (int i = 0; i < ndof; i++) {
            real_t s = 0.0;
            for (int j = 0; j < ndof; j++) {
                s += Minv_e[i*ndof + j] * rhs_local[j];
            }
            res_local[i] = s;
        }
        for (int i = 0; i < ndof; i++) {
            dQdt[c*ndof_total + e*ndof + i] = res_local[i];
        }
    }
});

```


- MAX_DOF_TET is computed from the maximum supported polynomial order via $(p+1)(p+2)(p+3)/6$. For supported orders $p \in \{1, 2, 3, 4\}$, MAX_DOF_TET = 35. (Production TPV/BP5 currently uses $p \in \{1, 2\}$; rarely $p = 3$ or 4.) The buffer can be enlarged to accommodate $p = 5$ (56), $p = 6$ (84), etc., only when verified in tests. All loops MUST be bounded by the actual runtime **ndof**, never by MAX_DOF_TET — using MAX_DOF_TET as a loop bound reads uninitialised stack memory.
- For $\text{ndof} > \text{MAX_DOF_TET}$, fall back to shared-memory tiling (defer to Phase 7). [R-GPU-003]

5. **ApplySpatialDerivative** kernel. Same shape as mass-inverse but with K_d instead of identity input:

```
mfem::forall(ne, [=] MFEM_HOST_DEVICE (int e) {
    for (int c = 0; c < NUM_STATE; c++) {
        // Compute Kd_Q[i] = sum_j K_d^e[i,j] * Q[c*ndof_total + e*ndof + j]
        // Then dQ_dx[i] = sum_j Minv_e[i,j] * Kd_Q[j]
    }
});
```

6. No race on **Mult**'s **dQdt**. Phase 3 leaves ComputeFaceFluxRHS on CPU but mass-inverse on device. The transition host write → device read is handled by dQdt.HostReadWrite() inserting a host→device copy. This is acceptable as a temporary bridge; Phase 4 removes it.
7. Memory budget. Per-element scratch QuadratureData for $p=4$ tet ($\text{ndof} = 35$, $\text{nqp} \approx 56$ for a 2^p+1 rule):
 - B: $\text{ndof} \times \text{nqp} \times 8 \text{ B} = 35 \times 56 \times 8 \approx 15 \text{ KiB}$ (shared across all elements).
 - G naively (per-element physical dshape table): $\text{ne} \times \text{ndof} \times 3 \times \text{nqp} \times 8 \text{ B}$ at 200m TPV102 with $\text{ne} \approx 1\text{M}$: $1\text{e}6 \times 35 \times 3 \times 56 \times 8 \approx 47 \text{ GiB}$ — too large for a single A100/M1250X.
 - Resolution: store G factored as $\text{dshape_phys}(i, d, q, e) = \sum_m \text{Jinv}(d, m, e) * \text{dshape_ref}(i, m, q)$ where Jinv : $\text{ne} \times 3 \times 3 \times 8 \text{ B} = 72 \text{ MiB}$ at $\text{ne} = 1\text{M}$, and reference dshape_ref : $\text{ndof} \times 3 \times \text{nqp} \times 8 \text{ B} \approx 47 \text{ KiB}$ (shared). Build the physical dshape on the fly inside the kernel via the small matvec. Document this in the QuadratureData header.

Interfaces

- New: `mfem::seas::kernels::ComputeVolumeRHSDevice(...), ApplyMassInverseDevice(...), ApplySpatialDerivativeDevice(...), BuildQuadratureData(...)`.
- New: `mfem::seas::kernels::QuadratureData` struct.
- New: `WaveOperator::InitDeviceBuffers()` (private), `DeviceIsActive()` (public, const).
- No public API change to `WaveOperator::Mult`, `AdvanceADER`, etc.

Edge Cases to Handle

- Heterogeneous mesh (mixed tet + hex) — Phase 3 requires uniform geometry. Abort if `fes_>GetMesh()->GetElementGeometry(0) != ...` varies across elements.
- Curvilinear mesh (high-order **Nodes**) — Phase 3 assumes a per-element-constant Jacobian (linear tets). Abort at `InitDeviceBuffers()` with:

```
MFEM_VERIFY(mesh.GetNodes() == nullptr ||
    mesh.GetNodes()->FESpace()->GetMaxElementOrder() == 1,
    "device path requires straight-sided tets - found "
    "curvilinear Nodes; per-QP Jacobian storage required");
```

Curvilinear support requires per-QP Jacobian storage; deferred to a future phase. [R-GPU-013]

- Element order > 4 — stack array MAX_DOF_TET = 35 would overflow if the kernel writes beyond **ndof**. Abort with `MFEM_VERIFY(ndof_per_el_ <= 35, "p>4 tet not supported on device " "path; raise MAX_DOF_TET and re-verify with shared-memory tiling")`. Production TPV/BP5 runs use $p \leq 2$. [R-GPU-003]
- First call from **Mult** on host (–device cpu) — `DeviceIsActive()` returns false, dispatch host body; `InitDeviceBuffers` is NOT called.

- Repeated **InitDeviceBuffers** calls — guarded by `device_buffers_initialized_`; second call is a no-op.
- Driver-owned ODE state vector residency — the input `Q` to `WaveOperator::Mult` is owned by the driver / `ODESolver`, NOT by `WaveOperator`. If the driver does not flip `state.UseDevice(true)` before `ode_solver->Init(state)`, every `Mult` call triggers a host→device copy of the full state ($\sim 73 \text{ MB} / \text{step} \times 4 \text{ RK stages} \times 1e4 \text{ steps} = \sim 3 \text{ TB}$ PCIe traffic over a 60 s run, often negating GPU speedup). The driver MUST mark `state.UseDevice(true)` once before the time loop starts when `Device::Allows(DEVICE_MASK)`. See Phase 3 “Files to Modify” drivers section below. [R-GPU-014]

Acceptance Criteria

- ☐ `test_wave_volume_device` passes: $\| \text{rhs_dev} - \text{rhs_host} \|_{\infty} < 1e-12$ over the entire RHS for TPV102 200m mesh on three random `Q` states (zero, plane-wave, exact-solution).
- ☐ `test_wave_mass_inverse_device` passes: same tolerance.
- ☐ `test_ader_spatial_derivative_device` passes: same tolerance for each direction.
- ☐ TPV102 1-s smoke (RK4 path) with `--device cuda` matches CPU run within $1e-10$ on stress / velocity probe outputs.
- ☐ Profile shows volume + mass-inverse kernel times are at least $5\times$ faster on GPU than CPU baseline for TPV102 200m mesh.
- ☐ TPV102 1-s run with `--device cuda` shows $< 5\%$ of wall time in host↔device transfers (via `nvprof --csv dtot/htot` rows or Nsight Systems CUDA HW timeline). Confirms R-GPU-014 driver contract is in place.

Dependencies

- Depends on: Phase 0.
- Required by: Phase 4 (shares `QuadratureData`), Phase 5 (uses `ApplySpatialDerivative`).

Phase 4: Wave-operator face-flux kernels on GPU

Goal

After Phase 4: `ComputeFaceFluxRHS` and `ComputeSharedFaceFluxRHS` run on the device for all face categories (interior regular / interior fault / boundary absorbing / boundary free-surface / mixed-flux / shared seam). The `dQdt.HostReadWrite()` bridge from Phase 3 is removed; the full `Mult` and `AdvanceADER` (minus ADER specific phases) run on device.

Files to Create

- `miniapps/seas/dynamic/kernels/wave_face_kernels.hpp/.cpp` — face kernels split into:
 - `ComputeInteriorFaceFluxDevice` (regular interior, no fault).
 - `ComputeFaultInteriorFaceFluxDevice` (interior fault faces).
 - `ComputeBoundaryFaceFluxDevice` (per-bdr-attr dispatch).
 - `ComputeSharedFaceFluxDevice` (ParMesh seam, with ghost data already exchanged).
 - `ComputeCentralFluxFaceDevice` (mixed-flux mode adjacent / continuous).
- `miniapps/seas/dynamic/kernels/face_table.hpp` — precomputed face bookkeeping (flat arrays, device-ready). NOTE on residency [R-GPU-009]: `mfem::Array<T>` does NOT have a `UseDevice(true)` method — that method is on `Vector`. For `Array<T>`, allocate with `Array<T>(n, mfem::Device::GetDeviceMemoryType())` and access from kernels via `array.Read()` / `array.Write()` / `array.ReadWrite()` (these route through the memory manager and trigger host→device on first device use). For `Vector` members, the existing `UseDevice(true)` + Read/Write pattern applies.

```

struct FaceTable {
    // Allocate each Array<int> with Device::GetDeviceMemoryType() –
    // do NOT call UseDevice on Array (method does not exist).
    Array<int> face_indices;           // mesh face id per bucket entry
    Array<int> elem1_indices;          // adjacent element index
    Array<int> elem2_indices;          // -1 for boundary faces
    // Vector members: marked UseDevice(true) after population.
    Vector normals;                    // n[q,d,f]
    Vector w_detJf;                    // face Jacobian weight
    Vector shape1;                     // shape on side 1: ndof_face*nqp*nf
    Vector shape2;                     // ndof_face*nqp*nf or empty
    // ... plus restriction maps to gather Q at face DOFs
};

```

- miniapps/seas/tests/unit/test_wave_face_flux_device.cpp — CPU/GPU equivalence per face bucket.
- miniapps/seas/tests/parallel/test_wave_face_flux_device_parallel.cpp — np = 2, 4 equivalence on shared seam.
- miniapps/seas/document/gpu_dev/CUDA_AWARE_MPI_NOTE.md — performance caveat document.

Files to Modify

- miniapps/seas/dynamic/wave_operator.hpp:
 - Add private members:


```

mutable kernels::FaceTable face_table_interior_;
mutable kernels::FaceTable face_table_fault_interior_;
mutable kernels::FaceTable face_table_boundary_;
mutable kernels::FaceTable face_table_shared_;
mutable kernels::FaceTable face_table_central_mf_;
void InitFaceTables() const;           // lazy, called from InitDeviceBuffers
          
```
- miniapps/seas/dynamic/wave_operator.inl:
 - **ComputeFaceFluxRHS** (line 2084): branch on DeviceIsActive():
 - * true → dispatch the four interior + boundary device kernels in sequence, each operating on its bucket.
 - * false → existing host body.
 - **ComputeSharedFaceFluxRHS** (line 2733):
 - * On device: dispatch the shared seam kernel after the ghost exchange. The shared kernel must read both local Q and the ParGridFunction::FaceNbrData() buffer (which lives in device memory if CUDA-aware MPI is enabled).
 - **Mult** (line 684):
 - * Remove the dQdt.HostReadWrite() bridge from Phase 3 (now everything stays on device).
- miniapps/seas/dynamic/godunov_flux.hpp/.cpp:
 - Tag every method of GodunovFlux with MFEM_HOST_DEVICE.
 - Move bodies inline into the header (or split to a _device.hpp sibling) so kernels can capture them.
 - Audit for std::sqrt, std::abs, std::max — all device-OK; reject any Vector/DenseMatrix method calls (replace with raw arrays).
- miniapps/seas/dynamic/fault_face_flux.hpp/.cpp:
 - Same MFEM_HOST_DEVICE pass for EvaluateADER, EvaluateADER_LSW, Evaluate. Have them call kernels::SolveSlipRateNRStableDevice from Phase 2 directly (no virtuals).
- miniapps/seas/dynamic/pml_layer.hpp/.cpp:
 - Tag ComputeDamping MFEM_HOST_DEVICE. Move the PMLayer::Dx/Dy/Dz static arrays into __constant__ memory (CUDA) or __device__ (HIP) via MFEM_CONSTANT_QUALIFIER.

Detailed Requirements

1. Bucket faces at construction. `InitFaceTables()` walks `fault_interior_faces_`, `fault_shared_faces_`, mesh interior faces minus fault, boundary faces grouped by `bdr_attr`, and `central_flux_face_set_` to produce one `FaceTable` per kernel bucket. Buckets must be disjoint — a face appears in exactly one table.
2. **FaceTable** precomputation. For each face:
 - Cache `Elem1`, `Elem2` indices and the face-local DOF restriction maps (which element DOFs lie on the face). MFEM's `L2FaceRestriction` can scaffold this for tensor elements; for tets, build manually using `mesh.GetFaceVertices + FiniteElement::GetFaceDofs` per face.
 - Cache `det(Jf) × ip.weight` and unit normal at each face QP via `mesh.GetFaceGeometricFactors(...)`.
 - For shared faces, also cache which side (`Elem1` vs `face-nbr`) is canonical “+”.
3. Interior face flux kernel. Pattern (pseudocode):

```
mfem::forall(nf_interior, [=] MFEM_HOST_DEVICE (int f) {
    const int e1 = elem1[f], e2 = elem2[f];
    real_t Qm[NUM_STATE], Qp[NUM_STATE];
    real_t flux[NUM_STATE];

    for (int q = 0; q < nqp_face; q++) {
        // 1. Evaluate Q^- (Elem1 side) and Q^+ (Elem2 side) at QP
        // 2. Build face-local rotation T from normal
        // 3. Call GodunovFlux::Evaluate(Qm, Qp, n, flux) (device-tagged)
        // 4. Scatter -w * shape * flux into rhs for both elements
    }
});
```

4. Fault interior face flux kernel. Same skeleton as (3) but calls `FaultFaceFlux::EvaluateADER` (which internally calls `kernels::SolveSlipRateNRStableDevice` from Phase 2). Also handles the canonical-frame swap via the precomputed `interior_fault_elem1_on_plus_` flag, exactly as the host code does.
5. Boundary face flux kernel. Dispatches by `bdr_attr` → face category enum:
 - Absorbing → `GodunovFlux::AbsorbingTotal(Q_self, Q_bg)`.
 - FreeSurface (Gamma) → `GodunovFlux::FreeSurfaceTotal`.
 - FreeSurface (Godunov) → `GodunovFlux::FreeSurfaceGodunovTotal`. These are precomputed per face into a `face_bc_type_Array<int>` so the kernel only branches on integer comparison.
6. Shared face flux kernel. Pre-requisite: ghost data must be present in `ghost_gf_→FaceNbrData()` before kernel launch. The kernel SHOULD consume `ghost_gf_full_state_→FaceNbrData()` (`vdim=NUM_STATE`, `byNODES`) — the R-1601 batched ghost GF — so a single `ExchangeFaceNbrData` call covers all 9 components in one MPI round. Reading the per-component `ghost_gf_` (`vdim=1`) from the kernel would require 9 separate exchanges per macro-step, 8× the MPI cost. Phase 4 explicitly relocates the macro-step path to consume `ghost_gf_full_state_`; the legacy `vdim=1` path (per the R-1504 deep-copy correctness note in `wave_operator.hpp:782`) stays on host until R-1504 is resolved. [R-GPU-010]
 - With CUDA-aware MPI: `ExchangeFaceNbrData` posts device→device sends/recvs; data is already device-resident.
 - Without CUDA-aware MPI: insert a device→host copy before `ExchangeFaceNbrData` and a host→device copy after. Mark with `// PERF[G-401]: requires CUDA-aware MPI to avoid this transfer`.
 - The `ParFiniteElementSpace` underlying `ghost_gf_full_state_` must be constructed with `Device::GetDeviceMemoryType()` (or its `FaceNbrData()` buffer must be re-allocated with that memory type at first device-kernel call) so the receive buffer lives in device memory.
7. No race on **rhs**. Each interior face writes into both `Elem1` and `Elem2` RHS slots. Two valid patterns:
 - A. Atomic adds via `mfem::AtomicAdd<real_t>` (declared in `general/backends.hpp:94`). Do NOT use raw `atomicAdd` — only `mfem::AtomicAdd<T>` is portable across CUDA, HIP, and the

OpenMP host fallback. On compute capability ≥ 6.0 (A100/V100) and MI200+, double atomics are fast enough; the `CUDA backends.hpp` provides a CAS-loop fallback for older architectures. [R-GPU-005]

- B. Two-pass colour sweep — partition faces into colours such that no two faces in the same colour share an element. Launch one forall per colour.
- C. Per-element gather — invert the loop: each element thread collects flux contributions from all its faces (requires element→face map). Phase 4 uses A (atomic) for correctness simplicity; Phase 7 may replace with B or C if profiling shows atomics dominate.

Interfaces

- New: `mfem::seas::kernels::FaceTable` struct (in `face_table.hpp`).
- New: `WaveOperator::InitFaceTables()` (private, called from `InitDeviceBuffers`).
- New: 5 kernel entry points (`ComputeInteriorFaceFluxDevice`, `ComputeFaultInteriorFaceFluxDevice`, `ComputeBoundaryFaceFluxDevice`, `ComputeSharedFaceFluxDevice`, `ComputeCentralFaceFluxDevice`).
- All `GodunovFlux::*`, `FaultFaceFlux::Evaluate*`, `PMLayer::ComputeDamping` become `MFEM_HOST_DEVICE`.

Edge Cases to Handle

- **MixedFluxMode** `!= None` — `central_flux_face_set_` overrides the default upwind for selected faces. `InitFaceTables` must split affected faces out of `face_table_interior_` and into `face_table_central_mf_`. Cache validity rule R-1408 documented at `wave_operator.hpp:153–161` still applies — re-call `InitFaceTables` if `SetMixedFluxMode` runs after construction.
- **UsePrecomputedFaceFluxes(true)** — TPV102 precomputed face-flux path uses a flat matrix-vector representation (already partially device-friendly). Phase 4 leaves this path on host for now; emit `MFEM_ABORT` if both `UsePrecomputedFaceFluxes(true)` and `DeviceIsActive()` are set. Future phase (post-7) can port.
- **FaultFrictionLaw::LSW** (TPV205) — `EvaluateADER_LSW` has no root finder; closed-form. Easier to port. Must dispatch correctly based on the `FaultFrictionLaw` enum value.
- **SubStep** side-channel (TPV104 substep iterator) — when `substep_I_imp_plus_flat_ != nullptr`, the fault branch consumes pre-computed imposed states instead of calling the friction solver. Device kernel reads the pointer pair; if both are device-resident, no change needed. The driver must allocate these buffers with `UseDevice(true)`. Phase 4 audits the driver's `SetSubStepFaultImposedStates` call sites.

Acceptance Criteria

- ☐ `test_wave_face_flux_device` (serial) passes: per-bucket CPU/GPU equivalence at $< 1e-11$ relative max-norm.
- ☐ `test_wave_face_flux_device_parallel` (np=2,4) passes at same tolerance, including shared seam.
- ☐ TPV102 1-s with `--device cuda`, full Mult on device, agrees with CPU run at $< 1e-10$ on all probe stations.
- ☐ TPV205 1-s (LSW) on device, full Mult, agrees with CPU at $< 1e-10$.
- ☐ TPV104 1-s (non-substep RK4 path) on device, full Mult, agrees with CPU at $< 1e-10$.

Dependencies

- Depends on: Phase 2 (friction NR kernels), Phase 3 (QuadratureData pattern + `DeviceIsActive()`).
- Required by: Phase 5 (ADER reuses the face kernels).

Phase 5: ADER predictor–corrector on GPU

Goal

After Phase 5: `AdvanceADER`, `ComputeADERTimeIntegrated`, `ComputeADERVolumeUpdate`, `ComputeADERFaceFluxRHS`, `ComputeADERSharedFaceFluxRHS`, `ComputeADERSubStepStates`, `EvaluateBulkAtFaultQPsCanonical` all run on device. TPV104 substep iterator path matches CPU output bit-for-tolerance.

Files to Create

- `miniapps/seas/dynamic/kernels/ader_kernels.hpp/.cpp` — ADER-specific variants of volume and face kernels accepting time-integrated state I and step size dt instead of instantaneous Q .
- `miniapps/seas/tests/unit/test_ader_predictor_device.cpp` — CPU/GPU equivalence for `ComputeADERTimeIntegrated` at order 2, 3, 4.
- `miniapps/seas/tests/unit/test_advance_ader_device.cpp` — equivalence for the full `AdvanceADER` step.
- `miniapps/seas/tests/parallel/test_substep_iterator_device.cpp` — TPV104 substep dispatch path on device.

Files to Modify

- `miniapps/seas/dynamic/wave_operator.inl`:
 - **`ComputeADERTimeIntegrated`** (line 1004): branch on `DeviceIsActive()` → call new `kernels::ComputeADERTimeIntegratedDevice` which encapsulates the Cauchy-Kovalevskaya recursion. Reuses Phase 3's `ApplySpatialDerivativeDevice`. Scratch buffers (`ck_D_curr_buf_`, `ck_D_next_buf_`, `ck_dQ_dxd_buf_`) marked `UseDevice(true)` at construction.
 - **`ComputeADERSubStepStates`** (line 1101): same branching pattern; uses the parallel `ck_substep*_buf_` scratch group.
 - **`ComputeADERVolumeUpdate`** (line 2057): forward to `kernels::ComputeVolumeRHSDevice` from Phase 3 (since Phase 3 verified `ComputeVolumeRHS` and `ComputeADERVolumeUpdate` share the same integrand).
 - **`ComputeADERFaceFluxRHS`** (line 3259): branch; device dispatch to a new `kernels::ComputeADERInteriorFaceFluxDevice` + `kernels::ComputeADERBoundaryFaceFluxDevice` + `kernels::ComputeADERFaultInteriorFaceFluxDevice` which differ from Phase 4 only by accepting I and dt instead of Q .
 - **`ComputeADERSharedFaceFluxRHS`** (line 4254): device dispatch + ghost exchange (same CUDA-aware-MPI caveat).
 - **`AdvanceADER`** (line 4717):
 - * Ensure `ader_I_buf_`, `ader_rhs_buf_` marked `UseDevice(true)` on first call (lazy init).
 - * The PML inline branch (line 4763) runs a separate per-element host loop. Implement `ONCE` as a shared free function in `dynamic/kernels/pml_kernels.hpp` (added in Phase 4):

```
void ApplyPMLDampingDeviceImpl(const Vector &input,
                               real_t bg_scale,
                               /* other args */,
                               Vector &rhs);
```

Phase 4's `ApplyPMLDamping` thin-wraps with `bg_scale = 1.0`; Phase 5's `ApplyPMLDampingADERDevice` thin-wraps with `bg_scale = dt`. The two paths must NOT have separate copies of the PML loop body — divergence between them is a known historical bug class. [R-GPU-016]
 - **`EvaluateBulkAtFaultQPsCanonical`** (line 1637): branch; device kernel `kernels::EvaluateBulkAtFaultQPsDevice` that reuses face table + canonical-frame swap.

Detailed Requirements

1. Scratch buffers. All 8 `ader*_buf_` and `ck*_buf_` members declared in `wave_operator.hpp`:659–666 become device-resident at first call (per R-1501 lazy-init contract). Add an explicit `buf.UseDevice(true)`; `buf.SetSize(N)`; sequence inside `InitDeviceBuffers`.

2. CK recursion correctness contract. The recursion at `wave_operator.inl:1034-1060` uses pointer-swap between `D_curr` and `D_next`. The device variant must keep the same swap semantics — implement as alternating `Vector *D_curr_ptr / Vector *D_next_ptr` outside the kernel launch; each kernel call reads from one and writes to the other.
3. Substep dispatch fidelity. Round-11 R-602/R-603 contract: when `substep_I_imp_plus_flat_ != nullptr`, fault face flux kernels consume pre-computed imposed states. Buffer alignment: `substep_n_total_fault_qps_ == GetNumTotalFaultQPs()`. Phase 4 device kernel already gates on this; Phase 5 audit confirms the driver allocates these buffers with `UseDevice(true)` and that `EvaluateBulkAtFaultQPsCanonical` writes to device-resident buffers.
4. Order-of-operations preservation. AdvanceADER order: CK predictor $\rightarrow$ Volume(I) $\rightarrow$ Face(I) $\rightarrow$ SharedFace(I) $\rightarrow$ PML(I) $\rightarrow$ MassInverse $\rightarrow$ $Q_{\text{new}} = Q + \text{rhs}$. Device path uses the SAME ordering to keep results aligned with the CPU reference.

Interfaces

- New: `kernels::ComputeADERTimeIntegratedDevice`, `ComputeADERVolumeUpdateDevice` (forwards to Phase 3), `ComputeADERInteriorFaceFluxDevice`, `ComputeADERBoundaryFaceFluxDevice`, `ComputeADERFaultInteriorFaceFluxDevice`, `ComputeADERSharedFaceFluxDevice`, `ApplyPMLDampingADERDevice`, `EvaluateBulkAtFaultQPsCanonicalDevice`.

Edge Cases to Handle

- ADER order 2 vs 4 — recursion depth differs; ensure the device loop handles $0 \in \{2, 3, 4\}$ without unrolling.
- `pml_layer_ == nullptr` — skip PML kernel launch (host path already does this).
- `has_bulk_bg_ == false` — fluctuation-Q semantics; PML damps to zero. Pass a zero `bulk_bg_` array (already in the host code).
- Sub-step pointer side-channel reset — driver calls `ResetSubStepFaultImposedStates()` between dispatches. Mutable null-pointer state must be re-checked on each ADER call.

Acceptance Criteria

- ☐ `test_ader_predictor_device` passes at orders 2, 3, 4 within $< 1e-12$ max-norm.
- ☐ `test_advance_ader_device` passes within $< 1e-11$.
- ☐ `test_substep_iterator_device` passes at `np=2` and `np=4`.
- ☐ TPV104 substep-iterator production run (60s wallclock target) with `--device cuda` agrees with the existing chunhui-benchmark baseline on max slip rate, rupture front timing, and final slip.

Dependencies

- Depends on: Phase 3 (volume + mass + spatial derivative), Phase 4 (face flux + face table).
- Required by: nothing.

Phase 6: BP5 RHS rebuild + traction recovery on GPU

Goal

After Phase 6: the BP5 quasi-dynamic time-step ODE-RHS function (`SEASQuasiDynamicOperator::Mult`) runs the RHS rebuild (`AssembleSlipContribution`, `AssembleDirichletLoading`) and traction recovery (`ComputeTraction` $\rightarrow$ `ComputeTractionAtQuadPoints` + `FaceQuadrature::GalerkinProject`) on the device when applicable.

Files to Create

- `miniapps/seas/domain/kernels/bp5_rhs_kernels.hpp/.cpp`:
 - `AssembleSlipContributionDevice(...)` — per-fault-face loop.
 - `AssembleDirichletLoadingDevice(...)` — per-bdr-face loop.
- `miniapps/seas/domain/kernels/traction_recovery_kernels.hpp/.cpp`:
 - `ComputeTractionAtQuadPointsDevice(...)`.
 - `FaceQuadratureGalerkinProjectDevice(...)`.
- `miniapps/seas/tests/unit/test_bp5_rhs_device.cpp`.
- `miniapps/seas/tests/unit/test_traction_recovery_device.cpp`.

Files to Modify

- `miniapps/seas/domain/elasticity_operator_assembly.inl`:
 - **AssembleSlipContribution** (line 420): branch on `DeviceIsActive()` → device kernel.
 - **AssembleDirichletLoading** (line 1040): branch → device kernel. Note: Dirichlet function evaluation happens at boundary QPs and depends on time t ; the device kernel takes pre-evaluated $g_D(x_q, t)$ values produced on host per step. (Time-dependent host evaluation is cheap and avoids pushing `std::function` to device.)
- `miniapps/seas/domain/elasticity_operator_traction.inl`:
 - **ComputeTraction** — branch; device kernels dispatch `ComputeTractionAtQuadPointsDevice` + `FaceQuadratureGalerkinProjectDevice` on a per-fault-face basis.
- `miniapps/seas/fault/face_quadrature.hpp`:
 - **GalerkinProject** (line 216): add a `GalerkinProjectDevice` overload mirroring the host body with `mfem::forall(num_fault_faces, ...)`.

Detailed Requirements

1. Reuse Phase 4 face table. BP5's fault faces are the same set as the dynamic-rupture interior fault faces from the physics-set perspective (both consume `FaultBasis`), but the two owning classes are separate — `ElasticityDomainOperator` does not contain a `WaveOperator`, so it cannot share the `WaveOperator` instance's `face_table_member`. Phase 6:
 - Promotes the `FaceTable` struct and the table-builder free function from `dynamic/kernels/face_table.hpp` (Phase 4) into a shared header `common/kernels/face_table.hpp`.
 - `ElasticityDomainOperator` gains its own mutable `kernels::FaceTable face_table_fault_`; private member, independent of any `WaveOperator` instance.
 - The free function `InitFaultFaceTable(const FESpace&, const FaultBasis&, const Array<int>& fault_face_indices, FaceTable&)` is called once at first device-kernel use, by both `WaveOperator::InitFaceTables` and `ElasticityDomainOperator::InitDeviceBuffers`.
 - Appendix C is updated to list both owners. [R-GPU-012]
2. Slip contribution RHS kernel — per-fault-face:
 - Read slip at QPs (already embedded into global 3D via `FaultBasis`).
 - Compute traction-test operator $[\sigma(\phi_{ke_p}) \cdot n]_u \cdot \delta_u$ at each QP.
 - Accumulate into element RHS via atomic adds.
3. Dirichlet RHS kernel — per-bdr-face (Dirichlet-tagged):
 - Read pre-evaluated $g_D(x_q, t)$ from a host-side scratch Vector marked `UseDevice(true)`.
 - Accumulate consistency + symmetry + penalty terms.
4. Traction recovery kernel — per-fault-face, port of `ComputeTractionAtQuadPoints` (line 434). Single `forall(f, nf_fault, ...)`; writes `traction_q` flat array of `dim × nqp_face × nf_fault` doubles.
5. L2 projection kernel — Galerkin projection (line 216 of `face_quadrature.hpp`) is per-face, small (`nbf × nbf × ncomp` linear solve where `nbf ≤ 15` for `p=4` tri). One `forall(f, nf, ...)`, register-resident matrix.

Interfaces

- New: 4 kernel entry points; 1 free function `InitFaceTables` in `dynamic/kernels/face_table.hpp`.

- `FaceQuadrature::GalerkinProjectDevice` (new overload).
- No public API change.

Edge Cases to Handle

- `use_petsc_ts == true` — PETSc owns the Mult callback signature. The state vector PETSc passes in must be device-resident; if PETSc was built without CUDA support, abort at startup (Phase 1 already added this gate).
- `solver_type == MUMPS` — already aborted in Phase 1 if device is active; no Phase 6 concern.
- Fault-face count = 0 on some ranks — `forall(0, ...)` is a no-op. MPI collective rules from R-1600 still apply for ghost exchange of slip / displacement.

Acceptance Criteria

- ☐ `test_bp5_rhs_device` passes within $< 1e-12$.
- ☐ `test_traction_recovery_device` passes within $< 1e-11$.
- ☐ BP5 500m 1000-step run with `--device cuda` matches CPU run at probe stations within $< 1e-9$ on slip and slip rate.
- ☐ Full BP5 100-year run with `--device cuda`: chunhui-benchmark PASS within existing 5% tolerance band.

Dependencies

- Depends on: Phase 1, Phase 2, Phase 4 (face table).
- Required by: nothing.

Phase 7: Profiling, validation, performance tuning, and INCITE readiness [V2]

Goal

After Phase 7: documented end-to-end speedups on Polaris (CUDA), Frontier (HIP), Summit (CUDA), and Aurora (CPU fallback); INCITE-grade scaling demonstration (weak + strong) on at least Polaris + Frontier; per-kernel performance baseline; regression test suite auto-running in CI; and a populated `INCITE_READINESS.md` deliverable suitable for inclusion in an INCITE proposal narrative. [V2]

Files to Create

- `miniapps/seas/tests/perf/perf_tpv102_device.cpp` — micro-benchmark:
 - 10 RK4 stages, no output.
 - Reports per-kernel time (Nsight / rocprof markers).
- `miniapps/seas/tests/perf/perf_bp5_device.cpp` — same for BP5.
- `miniapps/seas/tests/perf/scaling_tpv102.cpp` — weak + strong scaling driver: takes a `--np` and `--mesh-scale` argument, runs 10 macro-steps, reports per-rank wallclock + MPI communication fraction. Used by the INCITE readiness deliverable. [V2]
- `miniapps/seas/tests/perf/scaling_bp5.cpp` — same for BP5 quasi-dynamic. [V2]
- `miniapps/seas/document/gpu_dev/PERFORMANCE_BASELINE.md` — per platform table of per-kernel speedups, occupancy, register pressure, memory bandwidth on Polaris (A100), Frontier (MI250X per GCD), Summit (V100), Aurora (Xeon Max CPU). [V2]
- `miniapps/seas/document/gpu_dev/REGRESSION_CHECKLIST.md` — list of tests that must pass for any GPU-touching PR.
- `miniapps/seas/document/gpu_dev/INCITE_READINESS.md` — proposal-ready document containing: (a) single-node achievable performance per platform, (b) weak-scaling plots up to 1024 nodes per platform,

(c) strong-scaling plots at fixed problem size, (d) node-hour estimate justifying the requested allocation. Sourced from Appendix D of this plan. [V2]

- `miniapps/seas/document/gpu_dev/CI_MATRIX.md` — documents the required CI build matrix (CPU-only smoke, `--device debug` equivalence, optional GPU runner for `--device cuda`). Implementer must first determine which CI system this repo uses (`ls .github/workflows/ .gitlab-ci.yml .buildkite/ 2>/dev/null`), then wire the matrix into the appropriate config file. The plan does NOT pre-commit to GitHub Actions. [R-GPU-017]

Files to Modify

- `miniapps/seas/Makefile` — add `make perf` target that runs the micro-benchmarks and writes JSON output to `tests/perf/results/`. Add `make scaling` target that runs the scaling drivers (gated on `MFEM_USE_MPI=YES`). [V2]

Detailed Requirements

1. Profiling markers. Add Nsight / rocprof ranges around each device kernel call:
 - CUDA: `nvtxRangePushA / nvtxRangePop`, wrapped in `#ifdef MFEM_USE_CUDA`.
 - HIP: `roctxRangePush / roctxRangePop`, wrapped in `#ifdef MFEM_USE_HIP`.
 - Both ranges live in the same RAII wrapper class `mfem::seas::ProfilerRange(const char *name)`, so kernel-site code does not branch on backend.
2. Per-kernel timing collection. Use `mfem::tic() / mfem::toc()` pairs around each kernel launch. Aggregate over 100 steps to reduce noise.
3. CI matrix (separate from on-site INCITE runs):
 - CPU build: existing tests + `--device cpu smoke`.
 - CUDA build (on a local NVIDIA dev box): `--device debug` (host emulation) for correctness + `--device cuda` if a GPU runner is available.
 - HIP build (if AMD dev box is available): same, with `--device hip`.
4. Per-platform baseline targets. Targets come from the per-platform table in §“Numerical / performance constraints”. The Phase 7 deliverable confirms each row, OR adjusts the targets in the plan and re-runs (a target miss is data, not a failure). [V2]

Kernel	Polaris A100	Frontier MI250X (per GCD)	Summit V100	Aurora PVC
Friction NR loop	≥ 8×	≥ 6×	≥ 4×	Aurora gap
Volume RHS	≥ 6×	≥ 5×	≥ 3×	Aurora gap
Mass inverse	≥ 6×	≥ 5×	≥ 3×	Aurora gap
ADER predictor	≥ 5×	≥ 4×	≥ 2×	Aurora gap
Face flux (atomic)	≥ 4×	≥ 3×	≥ 1.2× (CAS)	Aurora gap
HYPRE	≥ 5×	≥ 4×	≥ 2×	Aurora gap
PCG+AMG	≥ 3×	≥ 2.5×	≥ 1.5×	Aurora gap
End-to-end BP5 500m	≥ 5×	≥ 4×	≥ 2×	Aurora gap
TPV102 1s				

5. INCITE Readiness deliverable (populates `INCITE_READINESS.md`):

- Single-node achievable: TPV102 200m, BP5 500m on each platform, 1 device (1 A100, 1 MI250X GCD, 1 V100, 1 Xeon Max socket). Report wallclock, memory peak, GPU utilisation %.
- Strong scaling: TPV102 200m at fixed problem size, $np \in \{1, 2, 4, 8, 16, 32, 64, 128\}$ on Polaris and Frontier minimum; plot speedup vs np and parallel efficiency.
- Weak scaling: TPV102 with mesh refinement keeping DOFs/rank constant, $np \in \{1, 4, 16, 64, 256, 1024\}$ on Polaris and Frontier minimum; plot time/step vs np .
- For Summit: same demos at smaller np (≤ 32) because Summit allocations are typically smaller.
- For Aurora (CPU fallback): same demos but `--device cpu` on Xeon Max sockets, np up to 16 nodes minimum. Flag the comparison-with-PVC gap explicitly in the readiness doc.

6. Document remaining gaps:

- UsePrecomputedFaceFluxes path (still host).
- SeisSol-style Fused-GEMM kernels (potential future via dFEM).
- Custom DG integrator PA path for BP5 stiffness assembly (still host).
- Aurora SYCL backend (mainline MFEM does not have one — see §“Aurora gap”). [V2]

Interfaces

- New: `mfem::seas::ProfilerRange(const char *name)` RAIL class wrapping NVTX / ROCTX ranges. [V2]
- Otherwise: nothing new at API level. Profiling is internal.

Edge Cases to Handle

- CI without GPU — `--device debug` runs MFEM's host-emulated device backend (memory protection on host pool), catches missing `Read()/Write()` syncs without requiring actual hardware.
- Multi-GPU MPI — `make scaling NP=2,4,8,16,32,64,128` runs strong-scaling sweep; documented in `INCITE_READINESS.md`. [V2]
- MI250X dual-GCD per device — Frontier scaling tables MUST report np in GCDs, not GPUs. 1 MI250X = 2 GCDs; an 8-GCD node has 4 physical GPUs. Mismatching units is a common reporting bug in INCITE proposals. [V2]
- Summit V100 atomic-CAS regression vs A100 — face-flux kernel on V100 may show ~5–10× slowdown vs A100 even though the rest of the operator is closer. Document this as a known cost in the baseline doc; do not “fix” by abandoning V100 — the demo still shows portability. [V2]
- Aurora PVC gap — Aurora rows in scaling tables are “CPU only” pending §“Aurora gap” resolution. Explicit caveat in the readiness doc text. [V2]

Acceptance Criteria

- ☐ `make perf` produces a JSON timing table on each available platform.
- ☐ `make scaling NP=...` produces strong + weak scaling JSON on Polaris + Frontier (minimum). [V2]
- ☐ `PERFORMANCE_BASELINE.md` is populated with measurements for Polaris, Frontier, Summit, Aurora (CPU). [V2]
- ☐ `INCITE_READINESS.md` is populated with single-node, strong, and weak scaling figures + a node-hour estimate matching Appendix D. [V2]
- ☐ All Phase 0–6 acceptance tests in the regression checklist pass on all four platforms (Aurora limited to CPU path). [V2]
- ☐ CI green on CPU build with `--device cpu` and `--device debug`.
- ☐ Per-kernel targets in the table above are met OR explicitly revised in `PERFORMANCE_BASELINE.md` with profile data justifying the new target. [V2]

Dependencies

- Depends on: Phases 0–6.
- Required by: INCITE proposal submission.

Dependencies

- Depends on: Phases 0–6.
- Required by: nothing.

PART II — Cross-cutting concerns

Testing Strategy

Phase	Test file	Type	Count	Verifies
0	test_device_init.cpp	unit	4	CPU/GPU configuration, repeated init guard
1	test_bp5_hypre_gpu.cpp	unit	3	Hypre PCG+AMG CPU/GPU equivalence, MUMPS abort gate
2	test_friction_device.cpp	unit	6	NR vs Brent, vector vs scalar, warm-start
3	test_wave_volume_device.cpp	unit	3	Volume kernel for {zero, plane-wave, exact} Q
3	test_wave_mass_inverse_device.cpp	unit	2	M^{-1} correctness
3	test_ader_spatial_derivative_device.cpp	unit	3	$\partial_x, \partial_y, \partial_z$
4	test_wave_face_flux_device.cpp	unit	5	5 face buckets
4	test_wave_face_flux_device_parallel.cpp	parallel	2	np=2, np=4 shared seam
5	test_ader_predictor_device.cpp	unit	3	ADER orders 2/3/4
5	test_advance_ader_device.cpp	unit	1	full predictor-corrector step
5	test_substep_iterator_device.cpp	parallel	2	np=2, np=4 substep dispatch
6	test_bp5_rhs_device.cpp	unit	2	slip + Dirichlet RHS
6	test_traction_recovery_device.cpp	unit	2	recovery + L2 projection
7	perf_tpv102_device.cpp, perf_bp5_device.cpp	perf	2	kernel timings JSON

Validation against reference: - Per-phase: chunhui-benchmark on BP5 100s short run and TPV102 1-s short run, compared to existing CPU golden. - Phase 5 closing: TPV104 60-s production benchmark vs Tandem / SeisSol reference (existing chunhui-benchmark infrastructure).

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Device::Configured called before Hypre::Init in some driver path	Med	High	Phase 0 grep audit of all drivers; static_assert ordering in seas::InitDevice body.

Risk	Likelihood	Impact	Mitigation
HYPRE built without GPU but --device cuda requested	High	Med	Phase 1 warning + silent CPU solve; explicit detection of GetHypreMemoryClass() == HOST.
Atomics on real_t slow on A100 / MI250X	Med	Med	Phase 4 uses atomics; Phase 7 switches to colour-pass / element-gather if profile shows > 30% time in atomics.
CUDA-aware MPI not available on target cluster	Med	High	Phase 4 provides device↔host bridge with a clear PERF [G-401] marker.
TPV205 Eval-uadeADER_LSW device port introduces sign-convention bug	Med	High	Test 4-bucket equivalence on TPV205 1-s in Phase 4 acceptance.
Per-element register pressure overflow at p > 4	Low	Med	Phase 3 aborts at ndof > MAX_DOF; Phase 7 shared-memory tiling path planned.
PETSc TS GPU integration mismatches MFEM device memory	Low	High	Phase 1 startup gate; Phase 6 acceptance test.
Mixed-flux path cache (central_flux_face_set_) stale after SetMixed-FluxMode	Low	Med	Re-honour R-1408 lifecycle contract; document re-call requirement.
Branch divergence in fault Brent solver	Med	Med	Phase 2 routes device to NewtonRaphsonStable (fixed iters).
Memory budget overflow at TPV102 100m + ADER order 4 + np=1	Med	Med	Phase 3 documents 12 GiB peak; production runs at np≥4 split state.
Bit-divergence between MUMPS (CPU) and AMG (CPU+GPU) drift over long BP5 runs	High	Med	Acceptance uses cg+amg on BOTH sides for apples-to-apples; long-time chunhui-benchmark accepts 5% envelope.

Risk	Likelihood	Impact	Mitigation
[V2] Aurora SYCL gap — no MFEM SYCL backend exists	High	High	Phase 0 ships a CPU-only <code>build_aurora.sh</code> ; INCITE proposal carries Aurora as “CPU readiness only” with a Plan-B note. Re-evaluate if MFEM upstream adds SYCL or if we author a <code>Backend::CEED_SYCL</code> bridge in Phase 8.
[V2] Summit V100 (CC 7.0) FP64 <code>atomicAdd</code> uses MFEM CAS-loop fallback	High	Med	Face-flux speedup target reduced to $\geq 1.2\times$ on V100 (vs $\geq 4\times$ on A100). Documented in §“Numerical / performance constraints” and Phase 7 baseline. Switch to colour-pass scatter (Phase 4 option B) is the principal mitigation if V100 face flux dominates wall time.
[V2] MI250X dual-GCD partitioning — reporting np in “GPUs” vs “GCDs” inconsistency	Med	Med	Phase 7 scaling acceptance criterion explicitly requires np reported in GCDs (1 MI250X = 2 GCDs; 8 GCDs/node = 4 physical GPUs). All Frontier plots in <code>INCITE_READINESS.md</code> use GCDs.
[V2] Per-site HYPRE-GPU version drift (Polaris vs Frontier vs Summit may carry different HYPRE micro-versions; AMG smoother / coarsening defaults differ across HYPRE 2.31, 2.32, 2.33)	Med	Med	Phase 1 acceptance loosened to “GPU and CPU AMG within $2\times$ iteration count, $< 1e-6$ L2 diff” already covers this; per-platform <code>PERFORMANCE_BASELINE.md</code> records the HYPRE version per site.

Out of scope (deferred to follow-on plans)

- Porting `UsePrecomputedFaceFluxes` to GPU (TPV102 alt path).
- Multi-GPU domain decomposition tuning (beyond Phase 7 scaling study).
- Hex re-meshing of BP5 / TPV* to unlock MFEM PA path (would require full geometry re-derivation; not pursued).
- AMR / moving-mesh GPU support (currently static-mesh only; R-1510).
- dFEM auto-diff path as alternative kernel generator (worth a side prototype but separate plan).

Appendix A: file-level inventory of changes by phase

Phase	Files created	Files modified
0	4	8
1	1	4
2	2	4
3	5	2
4	6	5
5	4	1 (large)
6	4	3
7	4	1
Total	30	28

Appendix B: kernel LOC budget

Kernel	Approx LOC	Notes
ComputeVolumeRHSDevice	80	Plus 40 for QuadratureData builder
ApplyMassInverseDevice	40	
ApplySpatialDerivativeDevice	60	
ComputeInteriorFaceFluxDevice	120	Atomics path
ComputeFaultInteriorFaceFluxDevice	200	+ canonical-frame swap
ComputeBoundaryFaceFluxDevice	150	4 BC types
ComputeSharedFaceFluxDevice	150	+ ghost data unpacking
ComputeCentralFluxFaceDevice	80	
ComputeADER* (6 variants)	600	Reuse via templates where possible
EvaluateBulkAtFaultQPsCanonicalDevice	100	
ApplyPMLDampingADERDevice	80	
SolveSlipRateNRStableDevice (+ vector)	80	Header-only inline
ComputeRHSDevice (friction wrapper)	70	
AssembleSlipContributionDevice	120	
AssembleDirichletLoadingDevice	100	
ComputeTractionAtQuadPointsDevice	180	
GalerkinProjectDevice	50	
Subtotal	~2260 LOC	
QuadratureData, FaceTable, init helpers	400	
Drivers + config + Makefile changes	200	
Tests (~17 test files × avg 250 LOC each)	4250	
Grand total	~7100 LOC	over 7 phases

Appendix C: kernel-level data residency contract

Buffer	Type	Owner	Resident on	Marked UseDevice(true)?	Phase
Q, dQdt (wave op state)	Vector	driver	both	yes — DRIVER MUST flip UseDevice(true) before ode_solver->Init per R-GPU-014	3

Buffer	Type	Owner	Resident on	Marked UseDevice(true)?	Phase
elem_mass_ivector	Vector	WaveOperator	device	yes	3
elem_Kd_packed[3]	Vector	WaveOperator	device	yes	3
ader_I_buf_Vector		WaveOperator	device	yes	5
ader_rhs_buf_, ck*_buf_					
QuadratureVector	{B, W, O, p, d, e, v, i, c, e, w, _, d, e, t, J}	WaveOperator	device	yes (each Vector via UseDevice(true))	3
FaceTable::Vectorials	{W, a, v, e, O, p, e, r, a, t, o, r, s}	WaveOperator	device	yes (per Vector)	4 / 6
	(Phase 4); Elasticity-Domain-Operator (Phase 6 — separate instance per R-GPU-012)				
FaceTable::Array	indices, elem1 indices, elem2 indices			does not have UseDevice; allocate with Array<int>(n, Device::GetDeviceMemoryType()), access via Array::Read() (R-GPU-009)	4 / 6
tau_pre_, slip_rate_, a_values, eta_values, depths, dc_values	Vector	FaultGeometry / RateStateFaultOperator	device	yes	2
evolution_kinematic scalar		RateStateFaultOperator	device (captured by value into kernel)	n/a	2
cached_Ah_	HypreParMatrix	ElasticityDomainOperator (HYPRE)	device (via HYPRE)	n/a (HYPRE manages via GetHypreMemoryClass())	1
B_, X_ (HypreParVector)	HypreParVector	ElasticityDomainOperator (HYPRE)	device (via HYPRE)	n/a	1
Slip RHS work vector (b_)	Vector	ElasticityDomainOperator	host (Phase 1), device (Phase 6)	Phase 6 onward	6
ghost_gf_full (R-1601 batched ghost GF)	ParGridFunction	WaveOperator	device	underlying ParFiniteElementSpace constructed with Device::GetDeviceMemoryType() (R-GPU-010); requires CUDA-aware MPI to keep FaceNbrData() device-resident during exchange	4

Buffer	Type	Owner	Resident on	Marked UseDevice(true)?	Phase
substep_I_innv*_flat_driver (TPV104 substep)	pointer to driver- owned buffer	driver	device	driver responsibility	5

Appendix D: INCITE node-hour budget [V2]

This appendix supports the §“INCITE Readiness deliverable” in Phase 7 and serves as the source for the budget figures we will reproduce in the INCITE proposal narrative. All numbers are planning ballparks, to be refined once Phase 7 produces single-node measurements.

Per-platform node-hour conversion

INCITE awards node-hours, not GPU-hours. One node-hour = all-devices-on-one-node-busy-for-one-wallclock-hour.

Platform	GPUs/node (effective compute units)	“1 node-hour” ≈
Polaris (A100)	4 GPUs	4 GPU-hours
Frontier (MI250X)	8 GCDs (= 4 physical MI250X)	8 GCD-hours
Summit (V100)	6 GPUs	6 GPU-hours
Aurora (PVC)	12 stacks (= 6 physical PVC)	12 stack-hours (CPU fallback uses 2 Xeon Max sockets ≈ 104 cores)

Development node-hours per phase (estimate)

These are dev-time runs needed to land each phase: short benchmarks, regression sweeps, parameter studies. They include re-runs after debugging.

Phase	Activity	Polaris	Frontier	Summit	Aurora (CPU)	Total dev
0	Build & smoke (1-s TPV102 + 10-step BP5 on each platform, ~50 attempts including build retries)	50 nh	50 nh	30 nh	30 nh	160 nh
1	BP5 HYPRE GPU solve (~200 cg+amg short solves at 500m)	200 nh	200 nh	100 nh	n/a	500 nh

Phase	Activity	Polaris	Frontier	Summit	Aurora (CPU)	Total dev
2	Friction NR kernel (~500 micro-tests + 50 BP5 100-step runs)	100 nh	100 nh	50 nh	n/a	250 nh
3	Volume + mass-inverse + ApplySpatialDerivative (~100 TPV102 200m short runs)	200 nh	200 nh	100 nh	n/a	500 nh
4	Face flux on GPU — hardest phase, expect ~500 short runs to debug bucket dispatch + atomics + shared seam	1000 nh	1000 nh	300 nh	n/a	2300 nh
5	ADER on GPU (~200 TPV104 substep runs)	500 nh	500 nh	200 nh	n/a	1200 nh
6	BP5 RHS + traction recovery (~100 BP5 1000-step runs)	300 nh	300 nh	100 nh	n/a	700 nh
7	Profiling + scaling (np $\in \{1,2,4,8,16,32,64,128,256\}$ weak + strong, several reps)	1500 nh	1500 nh	500 nh	300 nh	3800 nh
Dev subtotal		3850 nh	3850 nh	1380 nh	330 nh	9410 nh

Total development budget: $\approx 10\text{k}$ node-hours, dominated by Phase 4 (face flux) and Phase 7 (scaling demo). A reasonable ASK for the development portion of an INCITE proposal would be 15k–20k node-hours total across all sites (1.5 $\times$ safety factor + buffer for re-running after review fixes).

Production node-hours for the INCITE proposal

The INCITE proposal narrative needs to justify a production-run budget for the science the GPU port enables. Two example science campaigns:

Campaign 1 — BP5 parameter study (quasi-dynamic): - 12 parameter combinations ($V_p \in \{1e-12, 1e-11, 1e-10 \text{ m/s}\} \times W_f \in \{30, 40, 50, 60 \text{ km}\}$). - Each run: 100 simulated years, $\sim 10^7$ time steps at the current BP5 500m resolution. Expected wallclock at Polaris $\sim 3\times$ speedup vs CPU: ~ 24 hours on 32 nodes = 768 node-hours per run. - Total: 12 runs $\times$ 768 nh = $\sim 9.2k$ node-hours. Multiply by 2 for re-run / mesh-refinement budget $\rightarrow \sim 20k$ node-hours.

Campaign 2 — TPV104 high-resolution rupture sweep (dynamic): - 6 nucleation configurations $\times$ 3 mesh resolutions (200m, 100m, 50m). - Each TPV104 60-s run at 100m on 128 GCDs of Frontier with $\sim 4\times$ GPU speedup: ~ 6 hours wallclock = 96 node-hours. - The 50m resolution doubles per dimension $\rightarrow 8\times$ more DOFs, $16\times$ more wallclock per step (with smaller dt for CFL). Estimate ~ 1500 nh per 50m run. - Total: 18 runs at mixed res $\approx \sim 12k$ node-hours. With safety factor $\rightarrow \sim 25k$ node-hours.

Campaign 3 — TPV102 long-time validation (dynamic): - 4 ADER orders $\times$ 3 frictions (RS / FVW / LSW from TPV102 / 104 / 205 setups) $\times$ 2 PML widths. - 24 runs $\times$ ~ 200 nh each at production resolution $\rightarrow \sim 5k$ node-hours.

Production subtotal: $\sim 50k$ node-hours across the three campaigns.

Suggested INCITE ask

Combining dev + production, plus a 25% margin for unanticipated runs and Phase 7 baseline re-measurement:

Site	Suggested ask	Why this site
Polaris (ALCF)	25,000 node-hours	Primary CUDA dev + Campaign 1
Frontier (OLCF)	35,000 node-hours	Primary HIP dev + Campaign 2 (largest meshes, dual-GCD)
Summit (OLCF)	5,000 node-hours	Portability validation + Campaign 3 partial
Aurora (ALCF)	3,000 node-hours (CPU only initially; grow if SYCL gap closes)	CPU-fallback demonstration
Total	68,000 node-hours	

This is well within typical INCITE allocation sizes (which range from tens of thousands to ~ 10 million node-hours). The proposal narrative should: - Cite the dev budget separately from the production budget. - Acknowledge the Aurora gap explicitly and propose a separate Director's Discretionary or ASCR Leadership Computing Challenge (ALCC) ask if the SYCL gap closes mid-allocation. - Reference the §"Per-platform baseline targets" table in this plan as the readiness evidence.

Re-cost trigger

If Phase 7 single-node measurements show $\geq 2\times$ discrepancy from the speedup targets in §"Numerical / performance constraints", revisit this appendix and reset the production budget proportionally BEFORE submitting the INCITE proposal.